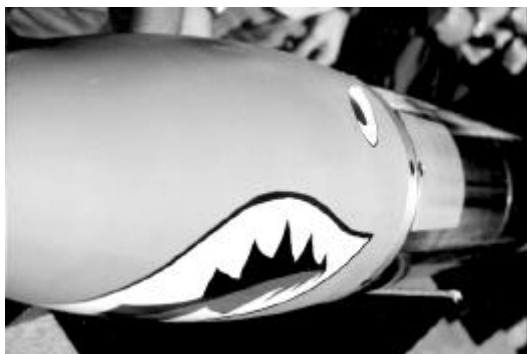




fusée expérimentale

B a b y l o n e

Rapport de projet

Présenté par le
Groupe Aéronautique
de l'Université Laval

Janvier 2000



Pour toute information sur le GAUL

GROUPE AÉROSPATIAL DE L'UNIVERSITÉ LAVAL

Département de génie mécanique
Pavillon Adrien-Pouliot
Casier 48
Université Laval
Sainte-Foy (Québec)
Canada G1K 7P4

Courrier électronique : gaul@gel.ulaval.ca
Page web : <http://www.gel.ulaval.ca/~gaul/>
Fax : (418) 656-7415 a/s M. Alain De Champlain

Directeur du projet *Babylone*
Florimond Laporte

Remerciements

Le GAUL tient à remercier tous ceux et celles qui, par leur aide financière, leur support technique, leur coopération ou leur appui, ont permis le lancement de la cinquième fusée expérimentale du groupe. Nous savons apprécier cette preuve de solidarité et nous vous invitons à continuer à appuyer les projets du GAUL: ils contribuent à chaque année à enrichir la formation de plusieurs étudiants en génie.

Principaux commanditaires

- ~~///~~ Centre de recherches pour la défense Valcartier (CRDV)
- ~~///~~ Agence spatiale canadienne (ASC)
- ~~///~~ Faculté de sciences et génie de l'Université Laval (FSG)
- ~~///~~ Département de Génie électrique et de Génie informatique de l'Université Laval
- ~~///~~ Département de Génie mécanique de l'Université Laval

Contribution financière

- ~~///~~ Confédération des associations d'étudiantes et étudiants de l'Université Laval (CADEUL)
- ~~///~~ Association des étudiants en sciences et génie de l'Université Laval (AESGUL)
- ~~///~~ Regroupement des étudiants de Génie informatique et électrique (REGIE)
- ~~///~~ Association des étudiants en Génie mécanique de l'Université Laval (AEGMUL)
- ~~///~~ Association des étudiants en Génie physique de l'Université Laval (AEGEP)

Contribution matérielle et appui technique

- ~~///~~ Centre national d'études spatiales (CNES)
- ~~///~~ Laboratoire de vision et systèmes numériques de l'Université Laval
- ~~///~~ Produits électroniques Elkel Ltée

Remerciements personnels

- ~~///~~ Alain De Champlain
- ~~///~~ Daniel Rousse
- ~~///~~ André Thibeault
- ~~///~~ Raynald Lacasse
- ~~///~~ Louise Laporte

Membres du Gaul lors du projet *Babylone***Directeur de projet :** Florimond Laporte**Directrice électronique :** Catherine Banville**Directeur mécanique :** Sébastien Poirier**Directeur commanditaires :** Alexis Lévesque**Trésorière :** Isabelle LaRoche

Patrick Dubois
Jean-Simon Bourgault
Louis Lamontagne
Steve Potvin
Stéphane Bureau
Laurent Leclerc
Simon-Pierre Harvey
Mathieu Fortin
Marc Duval
Pascal Gagnon
Jean-Sébastien Parent
Pierre-Yves Cloutier
Jean-François Bélanger
Benoît Lepage

Tâches des membres

Nom	Programme	Tâches
Florimond Laporte	GMC	Structure mécanique, parachutes, peinture.
Sébastien Poirier	GMC	Structure mécanique, système de récupération, gyromètre, caméra, assemblage mécanique.
Catherine Banville	GEL	Filtrage par traitement d'images avec FPGA, intégration de l'électronique.
Alexis Lévesque	GIF	Ordinateur de vol et code du HC11, séquenceur, panneau de contrôle, capteurs, télémétrie.
Patrick Dubois	GEL	Séquenceur et code du PIC, capteurs, ordinateur de vol, GPS, panneau de contrôle, rapport, télémétrie.
Jean-Simon Bourgault	GEL	Ordinateur de vol, GPS.
Louis Lamontagne	GMC	Pointe, tube de Pitot, peinture.
Steve Potvin	GIF	Page web, intégration.
Laurent Leclerc	GMC	Support électronique.
Stéphane Bureau	GIF	Code de l'ordinateur de vol, intégration.
Simon-Pierre Harvey	GPH	Caméra et système de rotation.
Mathieu Fortin	GPH	Caméra et système de rotation.
Marc Duval	GPH	Caméra et système de rotation.

Table des matières

Introduction et historique	1
1 Mécanique	2
2 Expériences.....	2
3 Électronique.....	3
3.1 Capteurs	3
3.1.1 Capteur de pression statique.....	3
3.1.2 Capteur de pression dynamique.....	4
3.1.3 Filtrage	5
3.1.4 Problèmes et solutions des capteurs de pressions.....	5
3.1.5 Accéléromètres.....	6
3.1.6 Gyromètres	7
3.1.7 États des actionneurs	7
3.1.8 Commentaires et recommandations sur les capteurs	7
3.2 Panneau de contrôle.....	8
3.2.1 Les modes de la fusée.....	8
3.3 Séquenceur	9
3.3.1 Principe de fonctionnement	9
3.3.2 Backup.....	10
3.3.4 Description de l'électronique	10
3.4 Télémésure.....	12
3.4.1 Pré-modulation	12
3.4.2 Émetteur	12
3.4.3 Antenne	12
3.5 Ordinateur de vol.....	12
3.5.1 Microcontrôleur	12
3.5.2 Interface du HC11 avec les autres composants de l'ordinateur de vol	13
3.5.3 Fonctionnalité supplémentaire du 68HC11 : un port série intégré.....	14
3.5.4 Structure de la mémoire et décodage d'adresses	14
3.5.2 Assignation des broches du 68HC11	14
3.6 Intégration et interconnexions	15
3.7 Recommandations sur l'électronique.....	16
4 La partie logicielle <i>Babylone</i>	18
4.1 Modes de la fusée.....	18
4.1.1 Mode normal (0000).....	18
4.1.2 Mode normal sans logique floue (0001).....	18
4.1.3 Mode test émetteur (0010).....	18
4.1.4 Mode test capteurs (0011).....	18
4.1.5 Mode test mémoire (0100)	18
4.1.6 Mode test logique floue (0101)	18
4.1.7 Mode récupération (0110).....	19
4.1.8 Mode reprogrammation (0111)	19
4.1.9 Mode démo port série (1000)	19
4.1.10 Mode démo (1111)	19
4.2 Tâches logicielles lors du vol.....	19
4.2.1 Les interruptions.....	19
4.2.2 La boucle principale	20
4.3 La logique floue	20
5 Résultats	22
5.1 Résultats des capteurs de pression	23
5.2 Résultats des accéléromètres.....	24
5.3 Données numériques	27
5.3 Données numériques	28
6 Conclusion.....	29

Introduction et historique

Le Groupe Aérospatial de l'Université Laval (GAUL) a été fondé en 1993; son objectif principal était et demeure toujours la conception et la réalisation de fusées expérimentales. Parallèlement, le GAUL participe également à diverses activités sociales dans le but de promouvoir le génie auprès des jeunes et du public en général. Depuis sa formation, le GAUL a réalisé sept projets dont six fusées expérimentales. La première de ces fusées, *Orion*, a été lancée en août 1994 au *Festival de l'Espace* de Bourges, en France. Ce festival est une campagne de lancement annuelle organisée conjointement par l'Association nationale science technique jeunesse (ANSTJ) et le Centre national d'études spatiales (CNES). *Orion* n'a pas été en mesure d'ouvrir le parachute qui lui aurait permis d'atterrir en douceur et d'être par la suite récupérée: le premier projet du GAUL a donc été un échec, imputable en majeure partie à l'inexpérience des membres pour qui la construction d'une fusée expérimentale était alors une activité nouvelle.

L'année suivante, le GAUL a conçu *Artémis*, une fusée très semblable à *Orion* mais dotée, entre autres, d'une électronique et de systèmes mécaniques plus performants et fiables. Son lancement en août 1995 au *Festival de l'Espace* de Bourges a cette fois-ci été un succès total: le système de récupération de la fusée ayant fonctionné de façon optimale, *Artémis* a pu être récupérée après avoir récolté les données fournies par ses différents capteurs. Cette réussite a poussé le GAUL à entreprendre, l'année suivante, un projet de plus grande envergure encore: il s'agissait de la fusée bi-étage *Apollon*, lancée en septembre 1997 à la base militaire de Valcartier. Ce projet s'est en effet étendu sur deux années compte tenu de sa complexité – celle-ci ayant peut-être été la cause de l'arrêt du signal de télémétrie après la séparation des deux étages qui a valu au projet la qualification de "succès en demi-teinte". Entre-temps cependant, le GAUL fournissait à l'automne 1995 la charge utile (des capteurs de pression atmosphérique) pour la fusée miniature *Capella* conçue et réalisée par l'Agence de recherche et développement de l'aérospatiale amateur (ARDAA). En 1996, l'arrivée de nouveaux membres a poussé le GAUL à entreprendre, simultanément au projet *Apollon*, la conception de la fusée expérimentale *Miranda*, qui a été lancée à Bourges en août 1997, un mois avant *Apollon*. *Miranda*, contrairement à *Apollon*, a connu un vol nominal avec une descente amortie par son parachute. Cependant, l'expérience principale qu'elle transportait, un gyroscope fabriqué par le GAUL, n'a pas fonctionné correctement. L'équipe du GAUL a tout de même remporté le prix Joseph Mercier pour son sérieux et son professionnalisme lors du lancement de *Miranda* au nouveau *Festival des Clubs Espace*, le nom de la campagne ayant été modifié en 1997.

En septembre 1998, *Riopelle*, la cinquième fusée du Gaul, fut lancée à Valcartier. Cette fusée ne comportait qu'un seul étage, contrairement à *Apollon*. *Riopelle* avait à son bord 2 fumigènes pour tracer la trajectoire de la fusée dans le ciel, comme des coups de pinceau, d'où le nom du peintre québécois. Malheureusement, le seul parachute n'a pas tenu le coup, *Riopelle* s'est donc écrasé avec fracas au sol. Heureusement, la télémétrie a fonctionné une partie du vol ce qui a tout de même permis d'obtenir des données durant le vol.

Finalement, le 11 septembre 1999 vers 15, *Babylone* fut lancée encore une fois à Valcartier. Elle atteint une altitude d'environ 1650 mètres avant de redescendre tout en douceur, ralentie par ses 2 parachutes. *Babylone*, a été un véritable succès pour le GAUL. Fonctionnant à merveille, elle fut la consécration d'une année d'efforts. On l'a récupérée sur le mont Corriano, tout près du site de lancement. Ce rapport traite donc en détail de la conception, de la réalisation et des résultats de la sixième fusée expérimentale du GAUL, *Babylone*.

1 Mécanique

2 Expériences

3 Électronique

Dans le cadre du projet *Babylone*, un système électronique assez complexe a été élaboré. Il est constitué de 4 cartes électroniques:

- ~~la~~ la carte des capteurs
- ~~la~~ la carte du panneau de contrôle
- ~~la~~ la carte du séquenceur
- ~~la~~ la carte de l'ordinateur de vol

Chacune de ces cartes a une dimension de 4,5" par 7". *Babylone* est également équipée d'un émetteur Toucan pour la télémétrie. Cet émetteur n'a pas été conçu par le Gaul, il est fourni par le CNES.

3.1 Capteurs

Les capteurs sont une fenêtre sur l'extérieur pour l'ordinateur de vol. Ils permettent de connaître différents paramètres tels que l'altitude, la vitesse, l'accélération ainsi que l'orientation de la fusée et l'état de ses actionneurs. La carte des capteurs de cette année est une amélioration de celle de l'an dernier. Les améliorations principales sont les nouveaux capteurs de pression de la série MPX5000 de Motorola ainsi que des accéléromètres *surface mount* ADXL250 de Analog Devices. De plus, la carte de cette année n'utilise aucun voltage négatif donc plus besoin d'un gros convertisseur DC-DC dispendieux.

3.1.1 Capteur de pression statique

L'altitude est une donnée vitale pour l'ordinateur de vol. C'est elle, en grande partie, qui détermine le moment de l'ouverture du parachute. Pour l'obtenir, une mesure de la pression ambiante est effectuée à l'aide d'un MPX5100AP de Motorola. Il s'agit d'un capteur qui peut mesurer des pressions variant de 15 à 115 kPa. Il fournit en sortie une tension variant de 0,2 à 4,7 V selon la formule suivante:

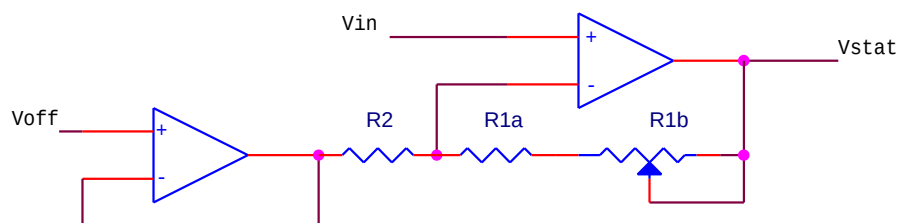
$$V_{\text{out}} = V_s(P \cdot 0,009 - 0,095)$$

où V_s est la tension d'alimentation de 5V

On a par contre relevé une caractéristique plus précise expérimentalement et on obtient plutôt la formule corrigée suivante:

$$V_{\text{out}} = V_s(P \cdot 0,00892 - 0,0906)$$

L'avantage principal de ce capteur par rapport à celui de l'an passé est que sa tension de sortie est déjà dans la plage 0-5V et qu'il est compensé en température. Malgré cela, il faut quand même un étage d'amplification pour faire un "zoom" sur la plage de pression qui nous intéresse. Cette plage a été déterminée entre 84 et 100 kPa (à tord, on en reparlera plus loin). Pour se laisser une marge d'erreur, on fixe comme critères une tension de sortie de 0,5V pour 84 kPa et de 4,5V pour 100 kPa. L'amplification est réalisée avec le circuit suivant:



V_{in} est la tension qui provient du capteur MPX5100AP et V_{off} est un voltage d'offset provenant d'un simple potentiomètre. V_{stat} est la tension de sortie presque finale qui passera dans un filtre Butterworth du 2^e ordre avant d'être lue par le HC11. La fonction de transfert de ce circuit est la suivante:

$$V_{stat} = \frac{(R2 + R1)}{R2} V_{in} + \frac{R1}{R2} V_{off}$$

Pour remplir la condition qu'on s'est fixée plus haut on a besoin d'un gain G de 5,698 et d'un offset de 18,333. On a en fait qu'une seule équation (celle du gain) avec 2 inconnues. On peut donc se permettre de fixer une des deux résistances à la valeur qu'on veut.

Il a été décidé de fixer $R2$ à 15 k Ω et de mettre un potentiomètre de 50 k Ω en série avec une résistance de 51 k Ω pour former $R1$. Avec ces valeurs on peut ajuster le gain entre 4,4 et 7,7. De façon générale, il est bon de rester avec des valeurs de résistances entre 10 et 100 k Ω pour les circuits d'ampli-OP.

Finalement, l'altitude de la fusée en fonction de la pression statique lue au capteur est la suivante (équation tirée du rapport d'Artémis) :

$$\text{Altitude(m)} = 9289,2 - (91,7) \cdot P \quad \text{où } P \text{ est la pression en kPa}$$

3.1.2 Capteur de pression dynamique

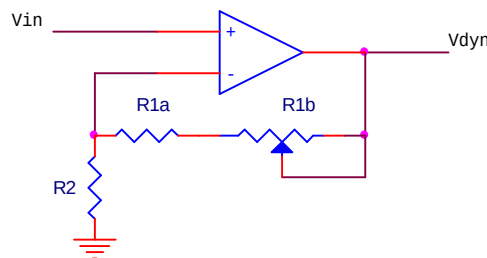
Pour mesurer la vitesse, un capteur de pression dynamique est utilisé. Il s'agit d'un MPX5050DP de Motorola. Il s'agit d'un capteur qui peut mesurer des pressions variant de 0 à 50 kPa. Il fournit en sortie une tension variant de 0,2 à 4,7 V selon la formule suivante:

$$V_{out} = V_s(P \cdot 0,018 + 0,04) \quad \text{où } V_s \text{ est la tension d'alimentation de 5V}$$

On a par contre, comme pour le capteur de pression statique, relevé une caractéristique plus précise expérimentalement et on obtient la formule corrigée suivante:

$$V_{out} = V_s(P \cdot 0,01826 + 0,0422)$$

On utilise un circuit d'amplification similaire à celui de la pression statique sauf qu'il n'y a pas d'offset.



V_{in} est la tension qui provient du capteur MPX5050DP et V_{dyn} est la tension de sortie presque finale qui passera dans un filtre Butterworth du 2^e ordre avant d'être lue par le HC11. La fonction de transfert de ce circuit est la suivante:

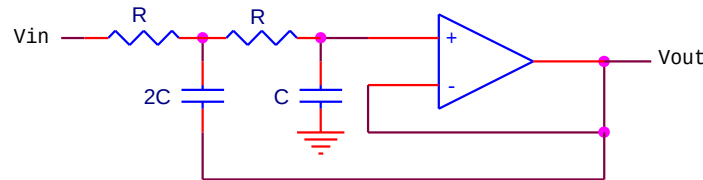
$$V_{dyn} = \frac{(R1 + R2)}{R2} V_{in}$$

C'est donc un simple amplificateur. La vitesse maximale de *Babylone* a été estimée **autour de 160 m/s** correspondant à environ 16 kPa. On a donc fixé comme condition d'avoir 4,5V en sortie avec 16 kPa. Cela

correspond à un gain désiré de 2,744. Pour ce faire, R2 a été fixée à 47 k Ω et R1 formé d'une résistance de 62 k Ω en série avec un potentiomètre multitours de 20 k Ω .

3.1.3 Filtrage

Le filtrage des signaux des capteurs de pression est effectué par un simple filtre Butterworth du 2e ordre. La fréquence d'échantillonnage étant de 100 Hz, la fréquence de coupure a été fixée à 50 Hz pour respecter le critère de Nyquist. ($f_c = f_e/2$). Le filtre est réalisé avec le circuit suivant :



La fonction de transfert de ce filtre est :

$$V_{out} = \frac{V_{in}}{s^2 + \sqrt{2} \omega_c s + \omega_c^2}$$

où la fréquence de coupure f_c se définit comme suit :

$$f_c = \frac{1}{2\pi} \frac{1}{RC} \quad f_c = \frac{1}{4\pi} \frac{\sqrt{2}}{RC}$$

3.1.4 Problèmes et solutions des capteurs de pressions

Les capteurs de pression de *Babylone* sont une amélioration par rapport à ceux de Riopelle mais ils sont encore loin d'être parfaits. Tout d'abord, il est très difficile d'ajuster les circuits d'amplification puisque l'offset et le gain dépendent des mêmes résistances. C'est donc dire qu'on ne peut pas ajuster le gain sans changer l'offset. Il serait grandement préférable d'avoir un premier étage d'offset suivi d'un étage d'amplification. De cette façon on pourrait ajuster ces deux paramètres séparément.

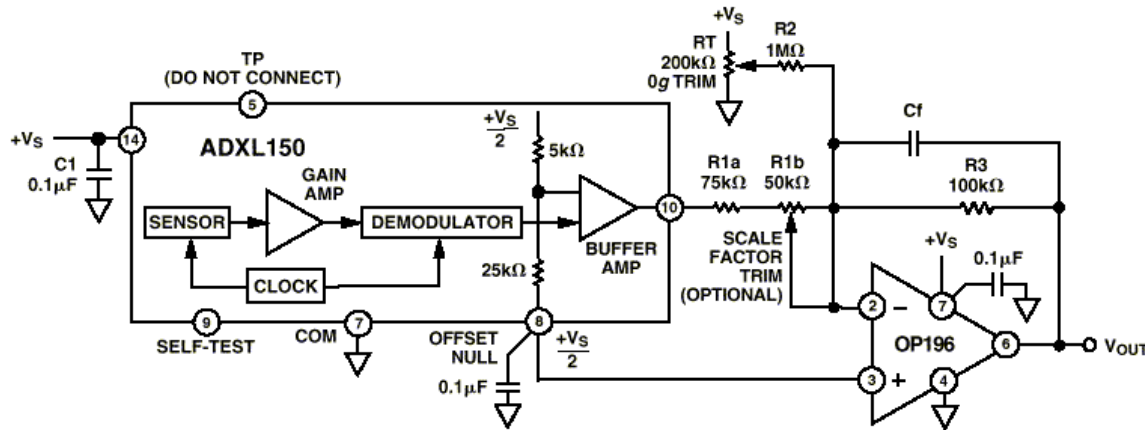
De plus, cette année, les deux capteurs de pression ont saturés, ce qui est totalement inacceptable! Le capteur de pression statique a saturé pour deux raisons, tout d'abord nous avons totalement ignoré l'altitude du site de lancement, en fait cette altitude de départ est loin d'être négligeable puisqu'elle se situe entre 200 et 300 m (donnée à vérifier ultérieurement). De plus, la pression atmosphérique n'est pas constante de jour en jour, si elle est plus basse que 101,3 kPa le jour du lancement, cette différence de pression est vue encore une fois comme une augmentation d'altitude. Il faudra donc penser à ces deux paramètres lors du choix de la plage de pression utilisée sur la prochaine fusée.

Le capteur de pression dynamique a saturé, lui, tout simplement à cause d'une sous estimation des performances de *Babylone*. En effet, le capteur a saturé à 170 m/s, une vitesse supérieure à celle qui avait été estimée. Il faudra donc être plus prudent l'an prochain et se laisser une marge de sécurité plus grande. Il faut toujours garder à l'esprit qu'il est difficile de prévoir précisément l'altitude d'une fusée expérimentale, surtout lorsque les performances du moteur peuvent être variables.

3.1.5 Accéléromètres

Les accéléromètres utilisés dans *Babylone* sont des ADXL250 de Analog Devices. Ils comportent deux axes à 90° qui peuvent lire des accélérations de $\pm 50g$. Pour pouvoir avoir une lecture sur les trois axes, il a fallu placer un accéléromètre perpendiculairement au circuit imprimé. Cela a été réalisé à l'aide d'un Surfboard, petite plaquette permettant de souder un chip surface mount dessus et de le placer à la verticale avec des pins thru-hole SIP (Single In-line Package).

Évidemment, $\pm 50g$ est une plage trop grande pour notre application, on a donc dû amplifier le signal pour aller chercher $\pm 10g$ avec les axes latéraux X et Y et $-5g$ à $+15g$ dans l'axe Z, celui de la fusée. Ces plages sont pour une excursion de tension de 0,5 V à 4,5 V, pour se laisser une certaine marge d'erreur. Le circuit d'amplification et de filtrage utilisé pour chaque axe est celui de la figure 20 du datasheet de l'ADXL250 disponible sur le site de Analog Devices.



Les valeurs de résistance sur ce schéma ne sont pas celles que nous avons utilisés. Pour les bonnes valeurs, se référer au schéma des capteurs à la fin de ce rapport.

Ce circuit d'amplification comporte un filtre passe bas de 1^{er} ordre avec le condensateur C_f . Un filtre de 1^{er} ordre semble suffisant puisque le ADXL250 comporte déjà un filtre passe bas à l'intérieur du chip lui-même. Pour une fréquence de coupure de 50 Hz, on doit avoir un condensateur C_f de 33 nF. On trouve cette valeur à l'aide de la formule suivante :

$$f_c = \frac{1}{R3 C_f}$$

La fonction de transfer du circuit d'amplification DC est la suivante :

$$V_{out} = \frac{1}{2} V_s \left(\frac{R3}{R1} \frac{V_{in}}{V_{adj}} \right) \left(\frac{R3}{R2} \frac{V_{adj}}{V_s} \right)$$

où V_{adj} est le voltage du potentiomètre RT sur le schéma. On détermine les valeurs des résistances en sachant que la sortie de l'accéléromètre est de 2,5 V à 0g et varie de 38 mV/g. Cette caractéristique de 38 mV/g n'est pas constante d'un accéléromètre à l'autre et ce n'est pas certain non plus qu'on aura 2,5V exactement à 0g, il faut donc faire l'ajustement du circuit d'amplification à l'aide des potentiomètres. On peut se référer au datasheet de l'ADXL250 pour cette procédure.

3.1.6 Gyromètres

Pour connaître l'orientation de la fusée, *Babylone* comportait 3 gyromètres fabriqués entièrement par le Gaul. Les circuits d'amplification qui apparaissent sur les plans sont pratiquement identiques à ceux utilisés pour les accéléromètres. La raison est qu'on croyait que le signal donné par les gyromètres serait environ de 2,5 V pour une position "zéro" et varierait positivement ou négativement suivant l'orientation de la fusée. Cette assumption était louable puisque la variation de position des gyromètres était faite grâce à un potentiomètre alimenté avec 5 V, dont on venait lire le voltage sur la *pin* du milieu.

En fait, le signal observé n'était pas rigoureusement 2,5 V à la position zéro et le circuit d'amplification était pratiquement inutilisable. Il a donc été décidé, à la toute dernière minute, de faire une simple amplification sans offset. Pour ce faire, toute filtration a été enlevée. Ce fut une erreur puisque notre amplification a fait saturer le signal sur 2 des 3 gyromètres. Vu le manque de temps, on aurait plutôt dû ne pas faire aucune amplification et simplement filtrer le signal.

3.1.7 États des actionneurs

Babylone est équipée de plusieurs détecteurs simples pour déterminer l'état de différentes composantes de la fusée. Pour chaque parachute, on a un détecteur d'ouverture de porte ainsi qu'un détecteur de déploiement. Pour le 1^{er} parachute, on a en plus un détecteur d'arrachement.

Détecteurs d'ouverture des portes

Le système de détection de l'ouverture des portes est réduit à sa plus simple expression: une résistance et un connecteur. Un bout de fil est fixé sur chaque porte et se débranche du connecteur à l'ouverture. Se faisant, la résistance n'est plus reliée à la masse et l'ordinateur de vol voit le niveau logique passer à 1.

Détecteur de déploiement du parachute

Le système de détection du déploiement des parachute est un peu plus complexe. Une cellule photo-électrique est placée au fond de la case de chaque parachute. Lorsqu'elle n'est pas illuminée, sa résistance est d'environ 100k Ω ; elle chute aux alentours de 2 k Ω lorsque la lumière du jour la frappe. Ceci fait varier la tension à ses bornes. Cette tension est comparée à une référence de 2.5V. Si elle est inférieure, soit non éclairée, le comparateur donne un niveau logique bas qui est transmis à l'ordinateur de vol. Lorsque le parachute se déploie, la résistance de la cellule chute, et la tension à ses bornes passe au-dessus de 2.5V. Le comparateur donne alors un niveau logique haut signifiant ainsi le déploiement du parachute à l'ordinateur de vol.

Détecteur d'arrachement du 1^{er} parachute

Ce système est identique au niveau électronique à celui pour la détection de l'ouverture de porte. La différence est que c'est un fil solidement attaché au parachute qui fait le lien entre la masse et la résistance. Si le parachute arrache, ce fil, bien que résistant, arrachera aussi certainement et on pourra ainsi agir en conséquence, soit déclencher l'ouverture du 2^e parachute immédiatement.

3.1.8 Commentaires et recommandations sur les capteurs

Les recommandations pour les capteurs de pression on déjà été données à la section 3.1.4. Il est quand même bon de se rappeler la recommandation principale, il faut se laisser une marge de sécurité plus grande! C'est une bonne chose d'essayer d'avoir le maximum de précision possible, mais ça ne donne pas grand chose quand les capteurs saturent!

Pour les accéléromètres, il faudra faire attention de bien fixer le SurfBoard au PCB avec de la colle. Lors de l'accélération de *Babylone*, une patte du SurfBoard n'a pas tenue le coup et a cédé, ce qui a rendu l'accéléromètre X inutilisable.

Pour ce qui est des photo-détecteurs de déploiement d'ouverture des parachutes, il faudra revalider le circuit. Il a été impossible d'ajuster un des photo-détecteurs le jour du lancement.

Finalement, autant que possible, il faudrait tester tous les circuits d'amplification de façon exhaustive avant de fabriquer les PCB. La carte des capteurs est celle qui a subi le plus de modifications cette année...

3.2 Panneau de contrôle

Le panneau de contrôle, comme son nom l'indique, sert à activer, programmer et tester la fusée. Il est le cœur de la fusée: c'est par lui que la plupart des signaux transitent. On y retrouve tous les interrupteurs d'alimentation (de l'ordinateur de vol, de l'émetteur, du séquenceur du parachute et du gyroscope), les interrupteurs de mode, le modulateur, tous les connecteurs d'alimentation et les connecteurs inter-cartes ainsi que le port série pour communiquer avec un ordinateur externe. Cette carte a été réalisée sur un circuit imprimé de 178 mm (7") par 114 mm (4.5").

3.2.1 Les modes de la fusée

Voici un résumé des modes ajustables sur le panneau de contrôle. Pour plus de détails concernant les modes de la fusée, consulter la section **4.1 Modes de la fusée**.

Combinaison (1 ^{re} étape)	Combinaison (2 ^e étape)	Mode
0000	1111	Normal
0001	1111	Normal sans logique floue
0010		Test émetteur
	0001	$F_{\text{mod}} = 15\text{kHz}$
	0010	$F_{\text{mod}} = 9\text{kHz}$
	0011	Onde carrée, $T = 2\text{ms}$
	Autres	Dent de scie
0011		Test capteurs
	0001	Pression statique
	0010	Pression dynamique
	0011	Accéléromètre X
	0100	Accéléromètre Y
	0101	Accéléromètre Z
	0110	Gyro (x)
	0111	Gyro (y)
	1000	Gyro (z)
	1001	Entrées numériques
	1010	Résultat de State_Read()
0100		Test mémoire
0101		Test logique floue
0110		Mode récupération
0111		Programmation EEPROM
1110		Démo port série
1111		Démo compteur

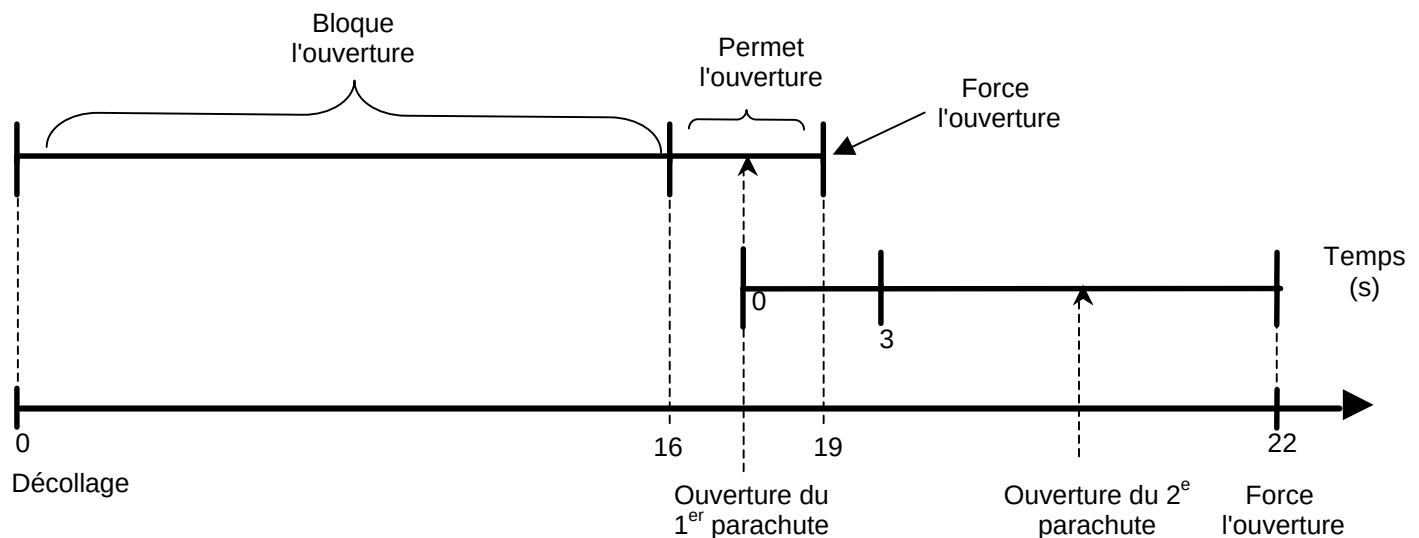
3.3 Séquenceur

Par où commencer... Le séquenceur est probablement, au niveau fonctionnement, la partie la plus compliquée de la fusée. En effet, le fait d'ajouter un 2^e parachute à la fusée complique énormément le séquenceur. Ce dernier n'a plus seulement à ouvrir un parachute, il doit maintenant "monitorer" ce qui se passe avec l'ouverture du 1^{er} parachute et agir en conséquence pour l'ouverture du 2^e.

Le séquenceur est la partie qui est responsable de l'ouverture des 2 parachutes de même que du déclenchement du fumigène. C'est une partie cruciale et c'est pourquoi le séquenceur est doté de sa propre alimentation et est isolé électriquement du reste de la fusée avec des opto-coupleurs. De plus, pour un maximum de sécurité il comporte un back-up pour l'ouverture du parachute principal, au cas où...

3.3.1 Principe de fonctionnement

Le séquenceur fonctionne selon le principe d'une fenêtre de temps double. La première contrôle le déclenchement du 1^{er} parachute et commence lors du décollage tandis que la deuxième contrôle le parachute principal et débute lors de l'ouverture du 1^{er} parachute. La figure suivante illustre ce principe:



On voit que ce n'est pas avant 16 secondes que le séquenceur permet l'ouverture du 1^{er} parachute. En effet, avant 16 secondes le PIC16C84 bloque tout signal provenant du HC11. Entre 16 et 19 secondes, le séquenceur permet au HC11, grâce à sa logique floue, de déclencher l'ouverture du 1^{er} parachute à l'apogée. Si, 19 secondes après le décollage, le PIC n'a toujours pas reçu de signal du HC11, il déclenche lui-même l'ouverture du 1^{er} parachute.

La 2^e fenêtre de temps débute lors de l'ouverture du 1^{er} parachute. Il y a un délai minimum de 3 secondes entre l'ouverture du 1^{er} et 2^e parachute. Pendant ce 3 secondes, le PIC bloque tout signal du HC11. Après ce délai de 3 secondes, le HC11 est libre de déclencher l'ouverture du 2^e parachute au moment opportun. Si, 22 secondes après le décollage le PIC n'a reçu aucun signal du HC11, il déclenche lui-même l'ouverture du 2^e parachute, le parachute principal.

Ce qui n'est pas bien illustré sur la figure est que la 1^{ère} fenêtre de temps est fixe alors que la position ET la longueur de la 2^e est variable. En effet, la position de la 2^e est déterminé par le déclenchement de l'ouverture du 1^{er} parachute tandis que la longueur, de même que sa longueur. La 2^e fenêtre de temps se termine toujours à 22 secondes, non pas après l'ouverture du 1^{er} parachute, mais bien après le décollage. Donc, le délai entre l'ouverture des deux parachutes sera entre 3 secondes et 6 secondes (si le 1^{er} ouvre à 16 secondes et le 2^e à 22).

3.3.2 Backup

Le backup, en principe, ne devrait pas servir si tout fonctionne normalement. Il est là seulement pour assurer une sécurité supplémentaire pour l'ouverture du parachute principal. Le cerveau du backup est un PAL16V8 qui implante une table de vérité. Le backup fonctionne avec deux timers. Le premier, qu'on appelle Tb1, passe de low à high à environ 16 s + 1 ou 2 s. Ce délai supplémentaire d'environ 2 secondes est pour laisser le temps au parachute de sortir de sa case avant d'observer le capteur de porte. Tbw quant à lui passe de low à high à 23 secondes, soit une seconde après la fin de la 2^e fenêtre de temps du PIC. Avant Tb1, le backup ne fait rien. Après Tb1, il vérifie si le drag est arraché, si oui, il ouvre aussitôt le parachute principal. Si après Tb1, le drag n'est pas ouvert (signal de porte pas déclenché) alors il ouvre également le parachute principal. Finalement, aussitôt que Tbw est atteint, la backup ouvre le parachute principal sans condition. Dans tous les autres cas, le backup ne fait rien et il laisse faire le PIC. La table de vérité dans le code du PAL16V8 en annexe aide à bien comprendre le backup.

3.3.4 Description de l'électronique

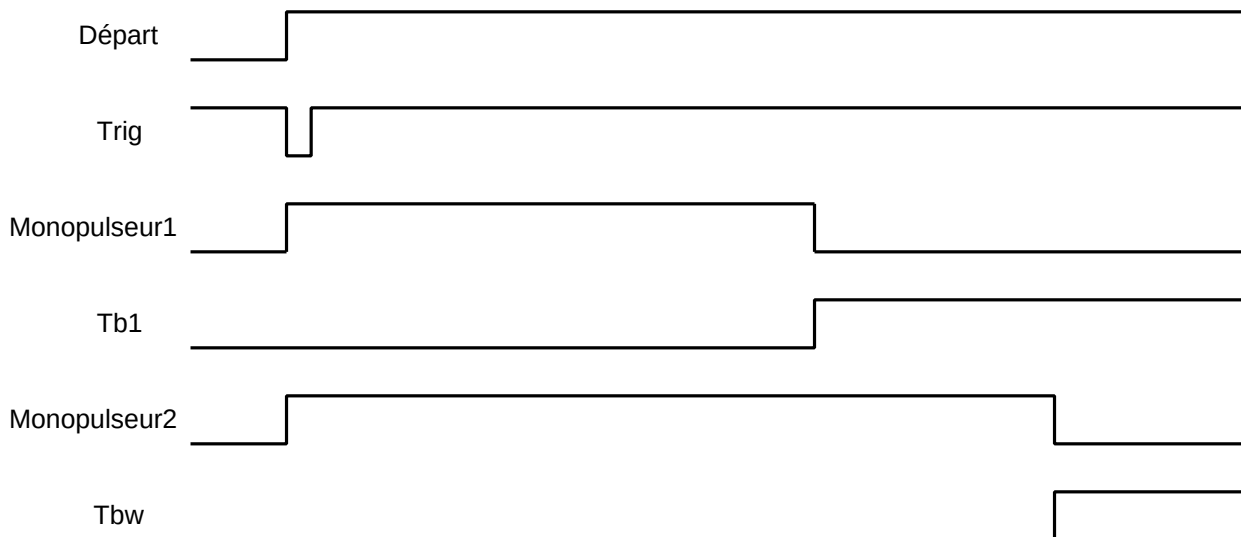
Le séquenceur est divisé en trois parties principales, le PIC, le backup et les sorties de puissance. Chaque partie a son propre ground et sa propre alimentation. Il y a un 4^e ground sur la carte, le ground de l'ovb. Pour isoler toutes ces parties, on utilise des opto-coupleurs HCPL-2730.

PIC

Le PIC16C84 est le cerveau du séquenceur, il contrôle toutes les sorties du séquenceur sauf la sortie du backup. Il est codé en C à l'aide du compilateur PCW de la compagnie CCS. Le code est inclus en annexe de ce présent rapport. C'est le PIC qui implante les 2 fenêtres de temps mentionnées plus haut. Chaque entrée du séquenceur provenant de l'ovb est isolée avec des opto-coupleurs. Les sorties du PIC sont également isolées des mosfets de puissance.

Backup

Le backup est plus complexe au niveau électronique que le PIC. Le plus difficile est de produire les signaux Tb1 et Tbw comme suit:



Pour générer Tb1 et Tbw, on utilise un LM555 utilisé en configuration monopulseur ainsi que de la logique à l'intérieur du PAL16V8. D'abord, le trigger qui déclenche les monopulseur, le signal Trig, est produit avec un petit circuit utilisant un mosfet et des condensateurs. Ce trigger est produit au décollage, donc lorsque le signal Départ

passé de low à high. Les signaux *Tb1* et *Tbw* ne sont jamais présents sur la carte, ils sont utilisés à l'intérieur du PAL et n'en sortent pas (sauf sur les pins 12 et 13 qui ne sont pas utilisés). Il faut faire attention, sur les plans, on voit *Tb1* et *Tbw* sur les pins 2 et 3 du PAL. En fait, ce sont les signaux *Monopulseur1* et *Monopulseur2* sur le timing diagram ci-haut. On peut déduire que la logique nécessaire pour obtenir *Tb1* est:

$$Tb1 = (\neg \text{Monopulseur1} \ \& \ \text{Départ}) \quad (\text{voir le code du PAL})$$

Tbw est obtenu de façon similaire.

La table de vérité suivante est celle qui est implantée dans le PAL. Le signal du départ est représenté par la lettre D, le signal d'ouverture de porte par P et le signal d'arrachement du 1^{er} parachute par A.

Tb1	Tbw	D	P	A	Out	
X	X	0	X	X	0	Pas décollé, on ne fait rien.
0	0	1	X	X	0	On n'a pas encore atteint Tb1, on laisse faire le PIC.
1	0	1	1	0	0	Tout va bien, drag ouvert.
1	0	1	1	1	1	Le drag est arraché, on ouvre le main!
1	0	1	0	0	1	Tb1 atteint mais le drag n'est pas ouvert, on ouvre le main!
1	0	1	0	1	0	Le parachute est arraché mais la porte n'est pas ouverte???
0	1	1	0	0	0	Tbw avant Tb1??? On ne fait rien.
0	1	1	1	1	1	Tbw avant Tb1??? Drag arraché? On ouvre le main!
0	1	1	1	0	0	Tbw avant Tb1??? Le drag est ouvert, parfait.
0	1	1	0	1	0	Tbw avant Tb1? Drag arrache mais porte fermée? Bizarre...
1	1	1	X	X	1	Adviennent que pourra, on ouvre le main!

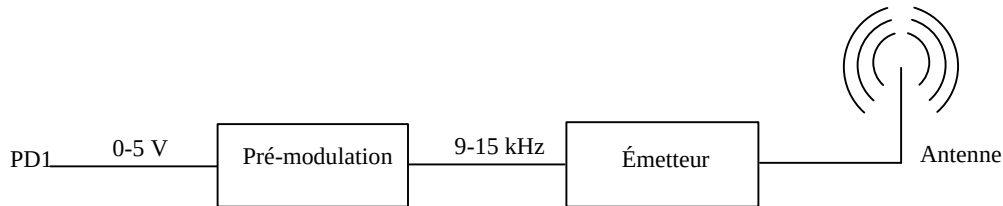
Finalement, pour produire les pulses nécessaires pour déclencher l'ouverture du parachute principal, on utilise un LM555 en mode astable connecté sur le PAL (voir le code pour plus de détails).

Étage de puissance

Chaque sortie du PIC et du PAL est reliée à un étage de puissance isolé qui fournit la puissance pour les solénoïdes et le fumigène. Sur les plans, on constate que chaque étage est très similaire. On note que sur 4 étage sur 6 il y a un opto supplémentaire. Cet opto-coupleur sert à retourner de l'information au HC11 sur l'état des sorties. On remarque aussi que sur les sorties des parachutes, il y a une diode en parallèle sur la sortie. Cette diode sert à encaisser les retours de courant du solénoïde. La raison pour laquelle l'étage de puissance est isolé du reste est que les solénoïdes faisaient planter le PIC. Même avec les diodes, le PIC plantait. Alors pour palier à ce problème, on a tout isolé avec des optos.

3.4 Télémésure

La fusée *Babylone* utilise la télémésure afin de transmettre, au récepteur se trouvant au sol, les données recueillies. La chaîne de télémésure à l'intérieur de la fusée, illustrée ci-dessous (Figure 3.1), comprend les éléments suivants: pré-modulateur, émetteur et antenne.



Chaîne de télémésure

3.4.1 Pré-modulation

La boîte de pré-modulation est un module servant à transformer le 0-5V venant du port série de l'ordinateur de vol en un signal de $5V_{pp}$ de 9 ou 15 kHz. Cette pré-modulation peut-être considérée aussi comme un convertisseur tension-fréquence. Pour réaliser ce petit module, un circuit intégré de type XR2206 est utilisé, car il s'agissait d'ajouter quelques résistances et condensateurs pour réaliser la fonction.

3.4.2 Émetteur

L'émetteur est l'élément qui module le signal à l'entrée, à la fréquence porteuse de 137,950 MHz. L'émetteur fournit ce signal à l'antenne avec une puissance de 300 mW. Un émetteur FM de type *Toucan*, fourni par le CNES, est embarqué dans la pointe de la fusée.

3.4.3 Antenne

L'antenne se situe à la fin de la chaîne de télémésure. Comme la configuration adoptée est un dipôle au-dessus d'un plan de masse, la longueur de l'antenne est de $\lambda/4$ soit environ 55 cm. De plus, l'antenne sert de tube de Pitot pour obtenir les pressions statique et dynamique.

3.5 Ordinateur de vol

L'ordinateur de vol est le cerveau de la fusée. Grâce à lui, les données des expériences peuvent être emmagasinées dans une mémoire et une partie de ces données peuvent être envoyées au système de télémésure pour une retransmission de l'information en temps réel au sol. Il est réalisé sur un circuit imprimé de 4.5 pouces par 6 pouces.

3.5.1 Microcontrôleur

Afin de réaliser toutes ses fonctions et de minimiser la circuiterie, le choix du micro-contrôleur s'est arrêté sur le MC68HC11A0 de Motorola. Bien qu'il ne s'agisse pas du microcontrôleur le plus puissant sur le marché, ses fonctionnalités répondent à nos besoins. Cette famille a l'avantage de contenir huit entrées analogiques distinctes menant, à l'aide d'un démultiplexeur, à un convertisseur analogique-numérique et une interface de communication

série asynchrone en plus d'un circuit de minuterie. De plus, il possède de la mémoire intégrée à même le boîtier. Pour ce qui est du MC68HC11A0, cette mémoire est composée de 256 octets de mémoire de type RAM. Tout ceci permet d'économiser temps, argent et espace.

3.5.2 Interface du HC11 avec les autres composants de l'ordinateur de vol

Évidemment, le 68HC11 ne pourrait être utile sans les circuits intégrés qui l'entourent. Voici donc un bref résumé des différents composants qui constituent l'ensemble de l'Ordinateur de Vol de *Babylone* (OVB).

Tout d'abord, la carte comporte une mémoire EEPROM 8 Ko permettant de stocker le logiciel qui guidera le comportement du HC11. Cette mémoire effaçable peut donc être reprogrammée à souhait afin d'effectuer des tests préliminaires au lancement. Pour améliorer les performances du système, une routine logicielle permet de relier directement un ordinateur personnel à l'OVB (via le protocole RS-232) pour reconfigurer la mémoire ROM. On évite ainsi d'aller configurer la mémoire sur un programmeur externe, qui n'est pas toujours à la portée de la main...

Ensuite, une mémoire statique non volatile est utilisée pour stocker les données recueillies par les expériences sur la carte des capteurs. Cette mémoire a une capacité de 512 Ko, permettant ainsi d'accumuler des données pour une période d'environ trois minutes si on tient compte de la quantité d'expériences dans *Babylone* et la fréquence d'échantillonnage qui est de 100 Hz. Cependant, un premier problème se pose ici : le 68HC11 ne possède que 16 lignes d'adresses alors qu'une plage mémoire de 512 Ko nécessiterait en théorie une étendue de 19 lignes d'adresses. Un tampon d'adresses hautes permet de résoudre ce problème. La technique illustrée ci-dessous utilise le même principe que la pagination mémoire. La solution choisie sépare l'espace mémoire en 32 pages de dimensions égales. Pour ce faire, il s'agit d'attacher les 14 lignes d'adresses les moins significatives du HC11 sur les bits les moins significatifs de la mémoire SRAM (A0-A13). Ensuite, il faut utiliser un circuit intégré incluant 8 bascules « D » (un tampon 74LS373) et brancher ses sorties aux adresses hautes de la mémoire (A14-A18). Ainsi, on traite les bits les plus significatifs de la mémoire indirectement et séparément des bits les moins significatifs. Les schémas électriques en annexe permettent de voir ces branchements. Ainsi, lorsqu'on veut inscrire une donnée en mémoire, il faut d'abord s'assurer que les lignes d'adresses hautes sont correctes et on peut ensuite accéder à la page qui a une grandeur de 16 Ko. Deux coups d'horloge sont donc nécessaires pour écrire en mémoire lorsque le tampon des hautes adresses doit être modifié.

La section traitant du décodage des adresses présentée plus loin donne un aperçu de la versatilité des nouveaux ordinateurs de vols conçus par le GAUL. Avec *Babylone*, la structure axée sur l'utilisation au maximum du bus d'adresses et de données permet d'écrire du code plus compact et plus aisément compréhensible. Ainsi, deux entités sur l'OVB se sont vues simplifiées grâce à ce bus de données :

- l'affichage hexadécimal présent sur le panneau de contrôle;
- la détection des entrées numériques (détecteurs d'ouverture de porte, d'ouverture de parachutes...).

Ces utilitaires font désormais partie de l'espace d'adressage de l'ordinateur de vol. Ceci permet donc de libérer quelques pattes du 68HC11 et de minimiser le nombre de manipulations à faire pour aller recueillir et afficher certaines données. Pour la détection des entrées numériques, un registre (tampon de 8 bascules D) agit comme interface entre les détecteurs eux-mêmes et le bus de données.

Cette année, un nouveau composant s'est ajouté au design. Il s'agit d'un contrôleur série de type UART permettant de générer deux ports série asynchrones distincts. Ce contrôleur de la compagnie Phillips s'est avéré nécessaire afin de rendre possible l'intégration du GPS à l'intérieur de la fusée. Il est doté d'une banque de registres facilement adressable par les bus d'un microcontrôleur comme le 68HC11 de Motorola. Comme le GPS n'a pas décollé à la base militaire de Valcartier, le UART ne sera pas traité en détail ici. Quelques recommandations seront présentées dans une section ultérieure.

3.5.3 Fonctionnalité supplémentaire du 68HC11 : un port série intégré

Par ailleurs, le 68HC11 de Motorola est doté d'un port série intégré utilisé à plusieurs sautes dans *Babylone*. Ce port série accepte et envoie des signaux de 0 ou 5 Volts avec le protocole série standard, incluant les conditions de départ et de fin d'octet. Cependant, il ne correspond pas tout à fait au standard RS232 qui, pour sa part, traite des signaux de 12 Volts. Le panneau de contrôle est donc doté d'un MAX232 qui a comme rôle de transformer le 0-5V en 12V. Par contre, le fait que le port série du HC11 utilise des niveaux logiques de 0-5V comporte ses avantages. En effet, nous pouvons utiliser ce même port série pour transmettre les données vers l'émetteur. Les détails sur la télémétrie seront donnés dans les sections subséquentes.

3.5.4 Structure de la mémoire et décodage d'adresses

Évidemment, pour un fonctionnement adéquat, tout ce beau monde se doit de respecter une certaine cohésion et une bonne synchronisation. Pour y arriver, tous les composants possèdent une ligne de contrôle permettant de les rendre actifs ou non (Chip Select). Comme dans les systèmes numériques standards, chaque composant possède sa plage d'adresse spécifique. Un décodage des adresses est donc nécessaire, et c'est là qu'on retrouve la plus grande amélioration par rapport aux ordinateurs de vol des fusées précédentes. Le décodage d'adresse est réalisé à l'aide d'un circuit intégré de type PAL. Évidemment, un tel système apporte beaucoup plus de souplesse qu'un décodage à base de portes logiques fixes sur la carte. Des erreurs de design (et il y en a eu...!!!) sont donc plus facilement rachetables puisque le PAL peut être reprogrammé sans modification à apporter au circuit. Voici donc l'espace d'adressage du cerveau de *Babylone* :

Adresses \$0000-\$00FF :	RAM du CPU
Adresses \$1000-\$103F :	Registres internes du 68HC11
Adresses \$2000-\$20FF :	Contrôleur d'interface série externe
Adresses \$3FE0-\$3FFF :	Tampon pour hautes adresses
Adresses \$4000-\$7FFF :	SRAM
Adresses \$A100-\$A10F :	Afficheur hexadécimal
Adresses \$A110-\$A11F :	Tampon d'entrées numériques
Adresses \$BFFF-\$FFFF :	EEPROM externe 8 Ko x 8 bits

Les intervalles qui ne sont pas spécifiés contiennent des adresses inutilisées. Dans ce cas, le PAL décodeur d'adresses garde la valeur de ses sorties aux valeurs par défaut. Ainsi, tous les composants sur la carte sont alors inactifs.

3.5.2 Assignment des broches du 68HC11

Pour *Babylone*, la façon de connecter les pattes du HC11 a changé considérablement par rapport aux autres années. Tout d'abord, l'utilisation d'un PAL décodeur d'adresses a permis d'intégrer plus d'informations sur le bus de données. Ainsi, le port A qui est considéré comme un port multi-usage s'avère donc moins utile, puisque presque toutes les entrées numériques sont accessibles via le bus de données. De façon générale, l'assignment des pattes est la suivante :

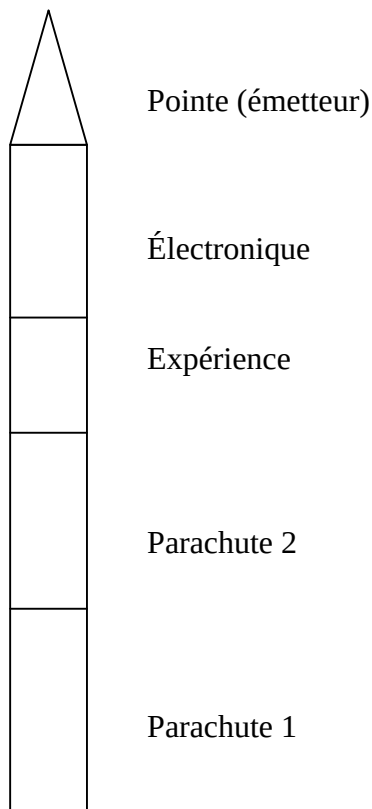
PORT A :	PA0	✂	Détection du décollage
	PA1 @ PA5	✂	Inutilisées
	PA6	✂	Commande du premier parachute
	PA7	✂	Commande du deuxième parachute
PORT B :	PB0 @ PB7	✂	Bus d'adresses

PORT C :	PC0 @ PC7	✍	Bus de données (et bus d'adresse multiplexé)
PORT D :	PD0	✍	RXD -- Port série : réception
	PD1	✍	TXD -- Port série : transmission
	PD2 @ PD5	✍	Mode de la fusée (où PD2 = 2^0 et PD5 = 2^3)
PORT E :	Convertisseurs analogiques-numériques		
	PE0	✍	Pression statique
	PE1	✍	Pression dynamique
	PE2	✍	Accéléromètre – axe X
	PE3	✍	Accéléromètre – axe Y
	PE4	✍	Accéléromètre – axe Z
	PE5	✍	Gyromètre – axe X
	PE6	✍	Gyromètre – axe Y
	PE7	✍	Gyromètre – axe Z

3.6 Intégration et interconnexions

Tel que spécifié dans les recommandations du rapport de Riopelle, la poutre électronique a été conçue de manière à ce que les cartes soient plus facilement accessibles. La méthode utilisée, soit des vis sans tête passant à travers la poutre, permet un accès rapide à toutes les cartes. Ceci étant dit, cette méthode fonctionne très bien.

En ce qui concerne les connexions inter-étages, maintenant, nous avons utilisé la même technique que l'an passé pour réaliser une fusée "plug 'n play".



On place un DB9 entre la pointe et la partie électronique, 2 DB25 entre la partie électronique et la partie expérience, un DB25 entre la partie expérience et la section du parachute 2 et un DB15 entre les deux sections parachutes.

Le principe est très simple. Tous les fils qui doivent faire le lien entre deux étages passent par les DB. Les fils sont regroupés dans une gaine qui est collée le long de la peau de la fusée.

Recommandations

La première recommandation, et la plus importante, est que l'intégration doit être faite au moins une à deux semaines avant le lancement. De plus, il serait bien de trouver un meilleur moyen que le "tape électrique" et la colle chaude pour maintenir les fils en place.

Ceci dit, le système est pratiquement à point et il est recommandé de le garder.

3.7 Recommandations sur l'électronique

Généralités

- ?? Mettre un jumper sur le write de la mémoire NVRAM (ou Flash) comme sur le EEPROM.
- ?? Mettre un pulse width modulator sur les afficheurs pour réduire la consommation de courant et pour ajuster leur intensité.
- ?? Mettre un bouton poussoir sur le panneau de contrôle pour permettre de déclencher l'ouverture de chaque parachute. Ne pas oublier de mettre un jumper de sécurité pour désactiver ces switches durant le vol!
- ?? Prévoir le connecteur spécial nécessaire pour brancher le GPS.
- ?? Essayer autant que possible de faire l'intégration de l'électronique minimum une semaine avant le lancement. Une journée n'est vraiment pas assez!
- ?? Penser à souder les fils sur l'antenne AVANT de mettre les tubes de plastique pour le Pitot.
- ?? Utiliser des spacers en plastique pour plus de sécurité contre les courts-circuits.
- ?? Laisser plus d'espace entre les cartes et la poutre électronique, il faut penser que les cartes peuvent plier légèrement et aller se court-circuiter sur la poutre.
- ?? Remonter le démodulateur avec un nouveau PCB s'il est réutilisé.
- ?? S'organiser pour synchroniser les pulses provenant du backup avec ceux du PIC pour le déclenchement du parachute principal (Un 555 dans l'étage de puissance...).

PCB

- ?? Mettre un "label" pour chaque jumper, indiquer sa fonction!
- ?? Indiquer la fonction des dip switches, avec tous les modes si possible.
- ?? Rendre les connecteurs de flat cable "fool proof" en enlevant une pin sur chaque connecteur et en bouchant la pin correspondante sur le connecteur femelle.
- ?? Mettre une petite led sur chaque board pour vérifier si le board est bien alimenté.
- ?? Mettre un carré pour la pin 1 des footprint des leds.
- ?? Ne pas mettre de test point s'il y a un jumper sur le signal!
- ?? Ne pas mettre tous les test points au même endroit, mais plutôt près du signal à prober.
- ?? Laisser un minimum de 4 pins libres sur chaque connecteur de flat cable pour laisser de la place pour d'éventuels signaux oubliés.
- ?? Router les condensateurs de découplage en premier et s'assurer qu'il sont bien connectés sur le bon chip.
- ?? Mettre des slide switches plutôt que des toggle switch, c'est beaucoup moins haut.

- ?? Essayer autant que possible de mettre tous les connecteurs au bout des cartes et non sur les côtés.
- ?? Pour les connecteurs de type "clip", utilisez ceux qui sont à 90°, c'est moins haut.
- ?? Changer le footprint ridicule des boutons reset.
- ?? Changer le footprint des TO-92 pour un footprint plus petit.
- ?? Favoriser les connecteurs flat cable et essayer d'éliminer autant que possible les borniers.
- ?? Faire des cartes de largeur 4" ou moins. Une largeur de 4½" cause des problèmes.
- ?? Faire un petit effort supplémentaire et identifier chaque signal sur les connecteurs sur les PCB, c'est pratique.
- ?? Penser à laisser de la place sur le PCB pour plier les mosfets (changer le footprint).
- ?? Faire un mini "verro board" sur chaque PCB si possible, ça pourrait éviter de coller des chips sur les PCB...
- ?? Laisser plus d'espace libre de traces autour de chaque trou de fixation.

4 La partie logicielle *Babylone*

Nous discutons ici de tout l'aspect logiciel de la fusée. Les modes de la fusée seront d'abord décrits un à un, puis les différentes tâches logicielles de l'ordinateur de vol seront énumérées. Cette section se terminera avec une description de l'algorithme de logique floue utilisé pour l'ouverture du parachute.

4.1 Modes de la fusée

Le micro-contrôleur de l'OVB doit accomplir plusieurs tâches distinctes et indépendantes. En effet, en plus du traitement principal des données lors du vol, il permet certains tests préliminaires ainsi que la récupération des données. En associant un mode d'opération pour chacune de ces fonctions, nous permettrons au 68HC11 d'exécuter une section de code différente pour chacune d'entre elles. Ainsi, avant d'effectuer un *RESET* sur le panneau de contrôle, il faut sélectionner le mode désiré à l'aide d'un *dip-switch* à quatre interrupteurs. Ces interrupteurs sont reliés aux entrées PD2, PD3, PD4 et PD5 du microcontrôleur. Leur état est lu à chaque fois que le système est initialisé. Voici donc la description de chacun des modes disponibles.

4.1.1 Mode normal (0000)

Ce mode regroupe toutes les opérations que le micro-contrôleur doit effectuer pendant le vol. Parmi celles-ci se retrouvent l'acquisition et le traitement des données ainsi que le calcul de l'algorithme de logique floue pour déterminer le moment optimal d'ouverture du premier parachute (le parachute de *drag*). De plus, l'octet décrivant l'état de la fusée apparaît sur l'afficheur hexadécimal du panneau de contrôle.

4.1.2 Mode normal sans logique floue (0001)

Ce mode est dérivé du mode précédent. La seule différence réside dans l'absence du calcul de l'équation de logique floue. L'ouverture des deux parachutes n'est commandé que par le séquenceur dans ce cas.

4.1.3 Mode test émetteur (0010)

Ce mode permet de tester l'émetteur de *Babylone* en envoyant un signal en dent de scie (onde triangulaire). Il permet également de calibrer et de pré-moduler le signal à envoyer, soit 9 ou 15 kHz.

4.1.4 Mode test capteurs (0011)

Ce mode nous permet de calibrer notre carte des capteurs. Une fois que ce mode est sélectionné, on peut choisir l'expérience à tester (pression statique et dynamique, accéléromètre, gyroscope et octet d'état), aussi à l'aide des interrupteurs. La valeur lue sur l'expérience en question (\$00-\$FF) est indiquée instantanément sur l'afficheur hexadécimal.

4.1.5 Mode test mémoire (0100)

Ce mode permet de tester la mémoire NVRAM de l'OVB. On y envoie un jeu de données connu et on vérifie la validité de chacune de ces données en allant la lire par la suite.

4.1.6 Mode test logique floue (0101)

Ce mode nous permet de tester la logique floue en envoyant sur le port série de l'OVB certains résultats intermédiaire de l'algorithme. On peut ainsi observer l'influence de chacun des capteurs sur le calcul menant au déclenchement de l'ouverture de la porte du premier parachute.

4.1.7 Mode récupération (0110)

Ce mode ordonne au 68HC11 d'envoyer sur le port série toutes les données contenues dans la NVRAM de l'ordinateur de vol. On procède ainsi à la récupération de résultats acquis par les diverses expériences.

4.1.8 Mode reprogrammation (0111)

Ce mode utilise le port série pour programmer le code qui doit être exécuté par le micro-contrôleur. Il nous permet donc de faire des modifications à notre code et de pouvoir reprogrammer le EEPROM rapidement, sans avoir à utiliser un programmeur d'(E)EPROM.

4.1.9 Mode démo port série (1000)

Dans ce mode, l'OVB transmet la valeur d'un compteur qui est continuellement incrémenté sur son port série (\$00 à \$FF). Ce n'est qu'une simple routine de test.

4.1.10 Mode démo (1111)

Ce mode affiche un décompte sur l'afficheur hexadécimal du panneau de contrôle. Cette routine permet de tester l'électronique de base de l'OVB ainsi que de vérifier la reprogrammation du EEPROM.

4.2 Tâches logicielles lors du vol

4.2.1 Les interruptions

Le programme de l'OVB fait appel à plusieurs routines qui sont appelées par des interruptions matérielles ou logicielles. Les deux principales interruptions qui ont été utilisées sont *TOC5_interrupt* et *SCI_interrupt*.

L'interruption *TOC5_interrupt* constitue la routine principale du code de l'ordinateur de vol pour l'acquisition des données. Elle survient à chaque fois que le contenu de TCNT (registre indiquant le temps déterminé par le compteur interne du 68HC11) est égal au contenu du registre TOC5. À chaque interruption, la valeur de ce dernier est augmentée de \$2710 (5 ms). De cette façon, une interruption est générée aux cinq millisecondes. Nous avons utilisé cette interruption pour procéder à l'acquisition des données. Une fois sur deux, l'OVB acquiert les données des capteurs de pression statique et dynamique, tandis que l'autre fois, il acquiert les données du gyroscope. Chacune des données est donc échantillonnée à une période de 10 millisecondes. La raison pour laquelle nous avons divisé l'acquisition des données en deux parties réside dans le fait que le 68HC11 ne peut effectuer que quatre conversions analogique/numérique à la fois, chacune d'entre elles entraînant un délai de 128 cycles. Un cycle correspond à une opération du micro-contrôleur et dure 500 ns. Ainsi, en effectuant huit conversions numériques, nous aurions attendu un temps non-négligeable à l'intérieur de la fonction appelée par l'interruption, ce qui est fortement déconseillé. Enfin, chacune des données recueillies, y compris l'octet d'état, est inscrite en mémoire. On envoie également quatre données (pression statique, pression dynamique, jauges de déformation et octet d'état) au sol, par voie hertzienne, à toutes les vingt millisecondes. Par ailleurs, l'interruption *SCI_interrupt* permet la transmission des données au sol. Elle est générée à chaque fois que le 68HC11 a terminé d'envoyer la donnée qu'il avait à transmettre sur son port série. Voici les données qui sont transmises à chaque appel de *SCI_interrupt* :

~~///~~ \$FF : caractère de synchronisation;

~~///~~ la valeur de la pression statique;

- /// la valeur de la pression dynamique;
- /// la valeur lue aux jauges de déformation;
- /// l'octet d'état.

4.2.2 La boucle principale

Mises à part ces interruptions logicielles, une partie du code de l'OVB se retrouve dans une boucle principale (de la fonction *main*). Cette section du programme réalise principalement les calculs nécessaires pour l'algorithme de logique floue qui détermine le moment optimal d'ouverture du petit parachute. En cours d'exécution (en mode normal), les opérations de ces quelques lignes de code sont interrompues principalement par *TOC5_interrupt*. Il n'en demeure pas moins que plusieurs calculs ont le temps de s'effectuer entre deux de ces interruptions.

4.3 La logique floue

Puisque les paramètres physiques caractérisant la trajectoire balistique d'une fusée expérimentale sont très approximatifs, il est risqué de commander l'ouverture des parachutes en ne se fiant que sur ces informations. Dans le cas de *Babylone*, nous avons procédé à quelques simulations afin d'estimer le temps nécessaire à la fusée pour atteindre sa culmination. Cependant, comme cette estimation n'est pas des plus précises, nous faisons appel à un algorithme de logique floue pour confirmer ce calcul. En effet, étant donné que la fusée est munie de capteurs de pression statique et dynamique, nous sommes en mesure de déterminer assez précisément le moment optimal pour l'ouverture des portes des parachutes. En fait, la logique floue commande l'ouverture du premier parachute (le *drag*) et le séquenceur se charge d'ouvrir le deuxième (le principal). Une équation qui tient compte de ces paramètres est utilisée pour déterminer cet instant. À chacun des paramètres est assigné un facteur de pondération qui peut varier pendant le vol. Cette méthode s'est avérée très efficace pour les fusées *Artémis* et *Miranda*, deux fusées conçues antérieurement par le GAUL.

L'équation utilisée pour la fusée *Babylone* se base sur cinq critères. La pondération associée à chacun de ces critères a pour objectif de traduire le degré de confiance que nous avons envers eux. Les éléments tenus compte dans la prise de décision sont la pression statique (altitude), la dérivée de la pression statique (vitesse verticale), la pression dynamique (vitesse totale), la dérivée de la pression statique (accélération) et le temps écoulé. Les facteurs les plus fiables sont bien entendus les dérivées respectives des deux pressions puisque ces résultats sont indépendants de la connaissance que l'on a de la trajectoire de la fusée. En effet, tout objet qui a une trajectoire balistique dans l'atmosphère observe une dérivée nulle de pression statique et dynamique à son point de culmination. Cependant, l'opération de dérivation a comme propriété d'amplifier le bruit sur un signal. C'est pourquoi nous prenons soin de filtrer les données, de façon analogique et numérique.

Voici donc l'équation de l'algorithme utilisé dans *Babylone* :

$$Fuzzy(P_s, P_d, DP_s, DP_d, t) =$$

$$\frac{1}{256} \cdot \frac{(255 \cdot P_s) \cdot Pond1 \cdot (255 \cdot DP_s) \cdot Pond2 \cdot P_d \cdot Pond3 \cdot (255 \cdot DP_d) \cdot Pond4 \cdot t / TEMPS_ESTIME \cdot Pond5}{1}$$

dans laquelle les diverses variables sont décrites ci-dessous.

P_s : pression statique filtrée. Elle est obtenue en faisant la moyenne gaussienne de dix valeurs de pression statique stockées dans un tableau de dix éléments. Cette moyenne gaussienne est définie par :

$$Gaussienne(P_{s1}, P_{s2}, \dots, P_{s10}) = \frac{1}{64} \cdot \sum_{n=1}^{10} (c_n \cdot P_{s,n})$$

avec $c_n = [1,2,3,4,7,8,10,12,12]$

DP_s : dérivée de la pression statique filtrée. Elle est définie par :

$$\begin{aligned} & \left| \frac{P_s(t) - P_s(t - dt)}{dt} \cdot Zoom1 \right| & \text{si } P_s(t) - P_s(t - dt) \geq 0 \\ & 0 & \text{ailleurs} \end{aligned}$$

La constante de dérivation dt est de 100ms et le facteur constant $Zoom1$ permet de remettre le résultat sur une plage de 256 bits.

P_d : pression dynamique filtrée. Elle est obtenue en faisant la moyenne gaussienne de la même façon que pour la pression statique, à l'aide de dix données.

DP_d : dérivée de la pression dynamique filtrée. Elle est définie de la même façon que la dérivée de la pression statique filtrée, sauf pour le facteur constant $Zoom1$ qui diffère dans ce cas.

t : temps écoulé depuis le décollage.

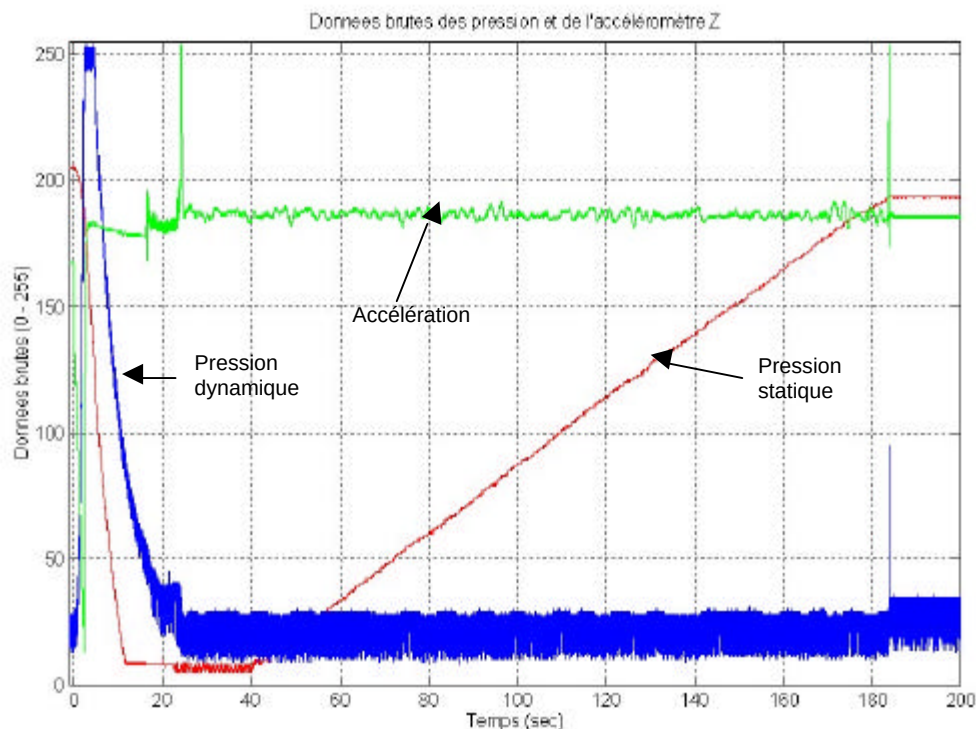
Cette équation est calculée à toutes les dix données de pression statique et dynamique. Si le résultat obtenu est supérieur à un certain seuil déterminé d'avance, la fonction *parachute_go()* est appelée. Si cette décision est approuvée par le séquenceur, le moteur qui permet l'ouverture de la porte du parachute est déclenché. Le seuil d'ouverture a été placé à 0,85 dans le cas de *Babylone*.

5 Résultats

Le lancement de *Babylone* a eu lieu avec succès à la base militaire de Valcartier le 11 septembre 1999 vers 15h. La fusée avait à son bord des capteurs de pression statique et dynamique, des accéléromètres, un gyromètre à trois axes, une caméra pour prendre des photos ainsi qu'un fumigène.

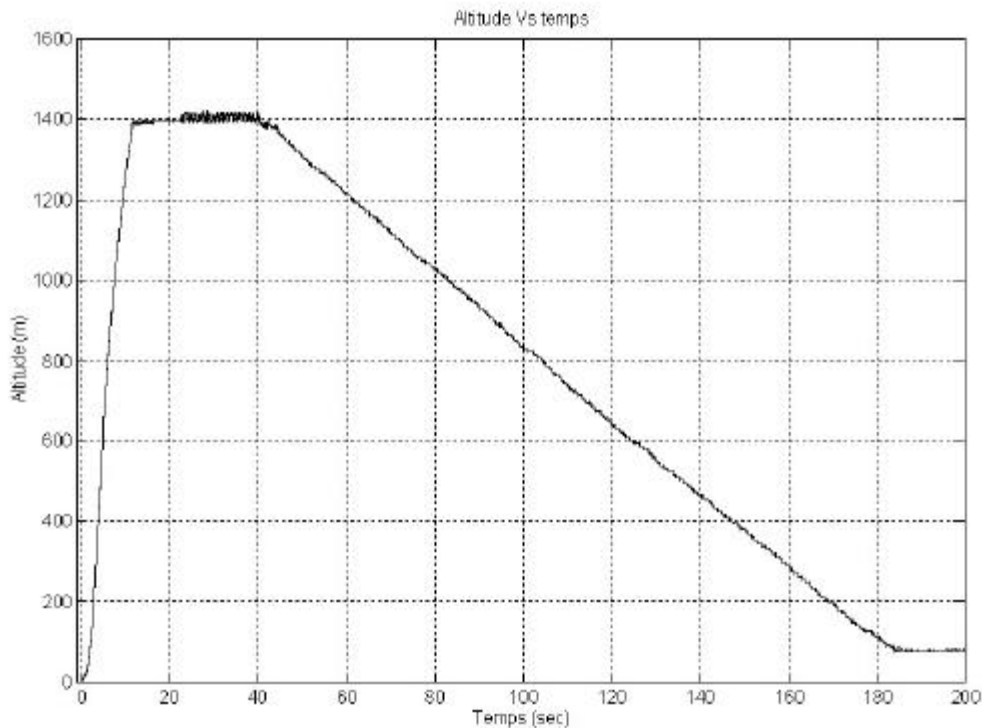
La mise à feu de *Babylone* a été parfaite. Les accéléromètres ont mesuré une poussée maximale d'environ 15 g d'une durée de 2,5 secondes. Les capteurs de pression ont fonctionné de façon nominale au début du vol, ce qui s'est gâté à la fin de l'ascension. Les mesures de pressions statique et dynamique ont respectivement saturé pour une vitesse de 170 m/s et une altitude de 1400 m. Cette saturation a provoqué l'ouverture du 1^{er} parachute par la logique floue au début de la fenêtre de temps, soit 16 secondes après le décollage. La sangle centrale du parachute de freinage s'est sectionnée à l'ouverture. Cette sangle assurait la forme du parachute et donc son efficacité - ou sa capacité à ralentir la fusée - vu la nature fermée du système (voir section des parachutes). Cependant, son sectionnement n'a pas semblé affecter le fonctionnement de l'ensemble, malgré les appréhensions. Une étude plus approfondie de ce type de ralentisseur permettra de mieux prévoir les impacts de tels événements. Le parachute principal s'est ouvert parfaitement à 22 secondes du décollage. Les accéléromètres ont mesuré une accélération négative de 7,5 g mais ils ont probablement saturé. On croit donc que la décélération que *Babylone* a subie est entre 8 et 9 g. La fusée est ensuite descendue tranquillement à une vitesse très constante d'environ 9,3 m/s jusqu'à ce qu'elle touche le sol, 183 secondes plus tard, à une altitude d'environ 120 m sur le mont Corriano. *Babylone* a encaissé un choc à l'atterrissage de 7 g, endommageant le tube de Pitot et la pointe en fibre de verre. La fusée a été conçue pour résister à un choc de 10 g à l'atterrissage, pour parer à une collision contre une surface parfaitement inélastique, une paroi rocheuse par exemple. Seulement, le mont Corriano jouit d'une végétation abondante, dont des arbres d'une dizaine de mètres, ce qui permet un amortissement efficace parce qu'étendue dans le temps.

Le graphique suivant présente les données brutes recueillies par les capteurs de pression et l'accéléromètre de l'axe Z.



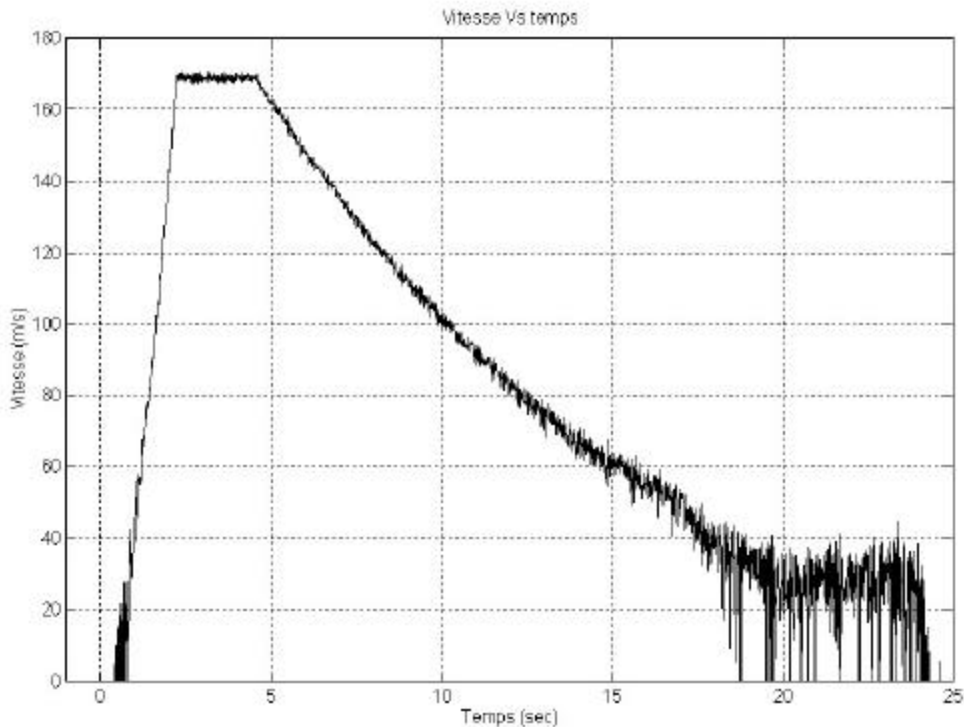
5.1 Résultats des capteurs de pression

Comme mentionné précédemment, les 2 capteurs de pression de *Babylone* ont saturé en vol. Malgré cela, les courbes permettent de bien cerner les caractéristiques du vol. Le graphique ci-dessous montre l'altitude de *Babylone* en fonction du temps. Comme la saturation a duré assez longtemps, il est difficile d'estimer l'altitude atteinte. On peut néanmoins extrapoler celle-ci à environ 1650 mètres.



Il est bon de rappeler ce qui a conduit à la saturation de ce capteur. L'erreur provient de l'oubli du zéro de la mer. Les capteurs de pression ont été étalonnés en considérant une altitude zéro au sol, ce qui n'est pas le cas à Valcartier. La pression au sol du champ de tir est relative à un niveau de 170 mètres au-dessus du niveau des mers. De plus, la pression du jour influence la lecture de l'altitude. En effet, puisque la pression baisse à mesure qu'on monte, si la pression du jour est inférieure à 101,3 kPa, cela a pour effet d'augmenter faussement l'altitude lue. Bref, l'altitude absolue qu'on lit avec le capteur ne vaut rien. Seule la différence de pression, et donc la différence d'altitude, mesurée par rapport à celle au sol est valable. Il faut donc enlever l'altitude initiale pour avoir l'altitude réelle. C'est ce qui a été fait pour obtenir le graphique ci-haut. Mais puisqu'on a retiré 290 mètres (!) pour avoir une altitude initiale de 0, on comprend vite pourquoi le capteur a saturé. On croyait que le capteur saturerait à une altitude de 1700 m mais en fait, il sature à 1400 mètres.

Le graphique suivant montre la vitesse mesurée à l'aide du capteur de pression dynamique :

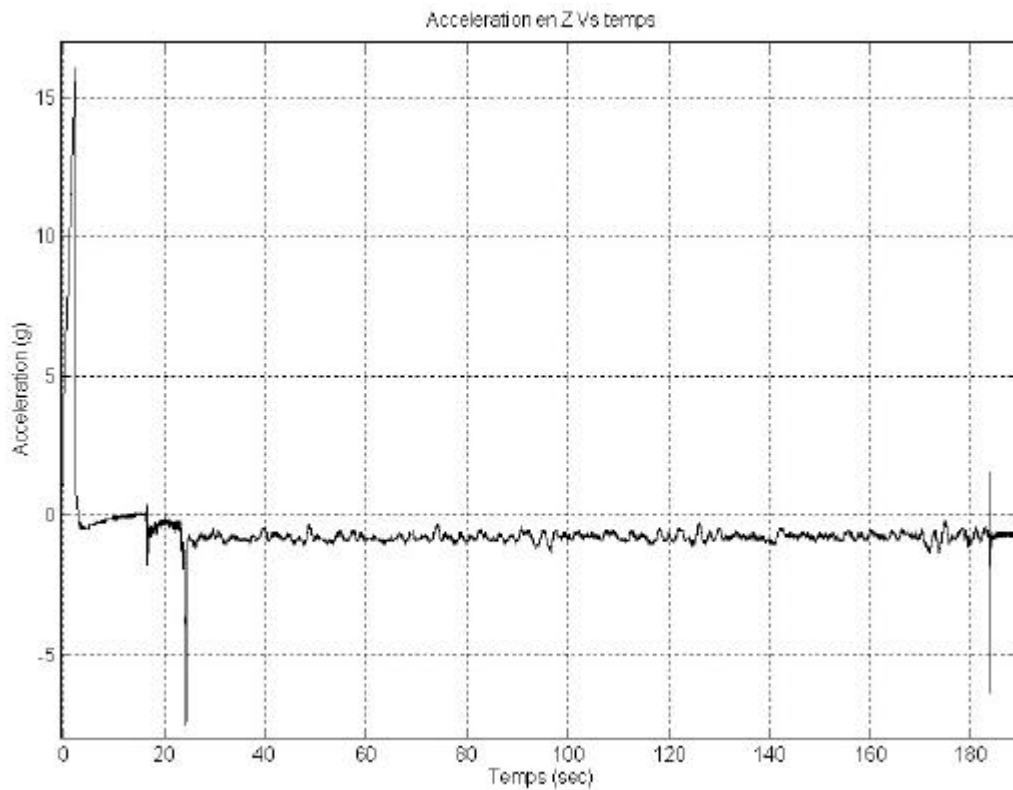


Le capteur de pression dynamique a lui aussi saturé, comme le montre la figure précédente. Moins d'informations ont été perdues par contre. Par extrapolation, on peut estimer une vitesse maximale d'environ 200 m/s ou 720 km/h. Pour une vitesse du son de 340 m/s, cela donne 0.58 M, un peu plus de la moitié de la vitesse du son, soit le domaine subsonique. Cette fois, la saturation du capteur est due à une sous-estimation des performances de *Babylone*. Les facteurs en jeu oscillent entre une mauvaise estimation de la masse finale, des caractéristiques imprévues du propulseur et une estimation erronée des paramètres de vol.

1. Plusieurs ajustements de dernières minutes peuvent influencer la masse finale. L'ajout ou le retrait de piles, d'éléments défectueux, ou même le perçage imprévu peuvent tous faire varier la masse.
2. Il faut savoir que les propulseurs utilisés proviennent de systèmes existants et modifiés pour l'occasion. Il est donc possible que d'une fois à l'autre, les modifications ne soient pas exactement les mêmes. Le propulseur n'étant pas sous la gouverne directe du GAUL mais sous celle du CRDV, il est difficile pour le groupe de connaître précisément ses caractéristiques.
3. L'estimation précise et réaliste de la vitesse et de l'altitude dans le temps est une des parties difficiles de la réalisation des fusées expérimentales. Il existe en effet plusieurs modèles mathématiques permettant de simuler un vol mais ils sont en général incomplet, c'est-à-dire qu'ils ne tiennent pas compte de tous les paramètres de vol. Le coefficient de traînée réel (C_d) est en réalité difficile à évaluer et son intégration au modèle mathématique est souvent erroné du fait que le C_d varie en fonction de la vitesse. L'écart entre la réalité et les prévisions réside donc dans ces mauvaises évaluations.

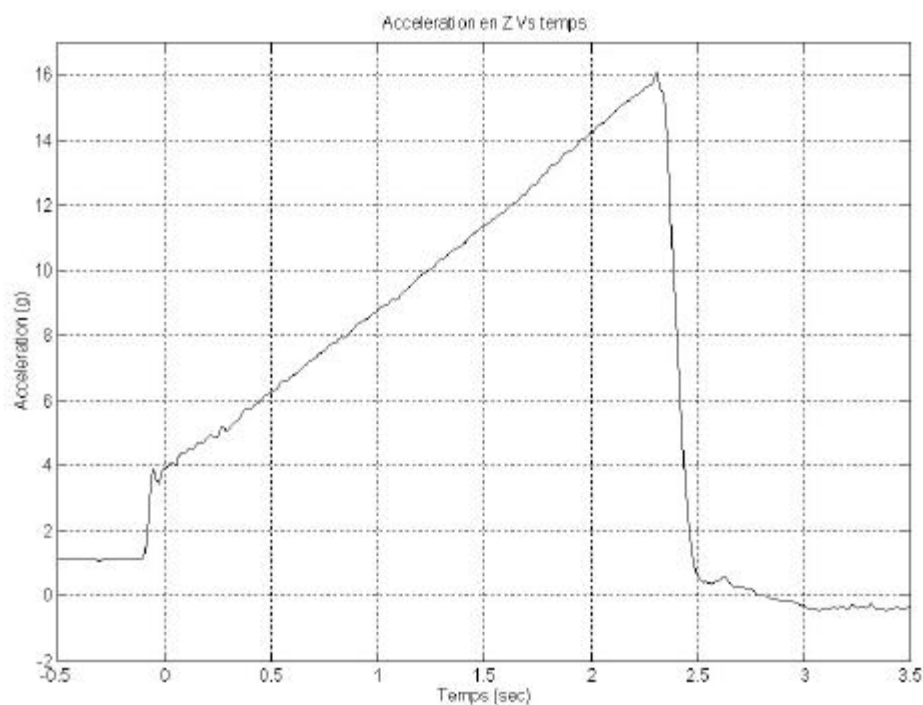
5.2 Résultats des accéléromètres

Babylone avait à son bord 3 accéléromètres, soit un pour chaque axe de l'espace cartésien, l'axe Z étant longitudinal à la fusée. Les résultats des accéléromètres sont très intéressants.



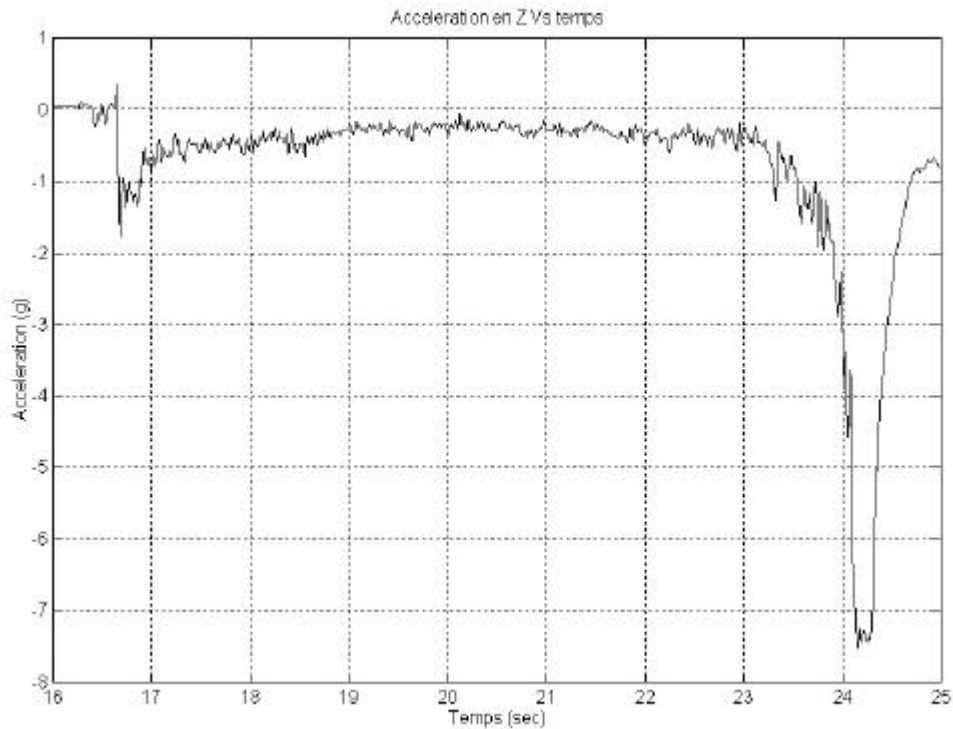
Commençons par un graphique de l'accélération en Z durant la durée complète du vol. On peut voir la forte accélération au décollage, l'ouverture de chacun des parachutes et l'atterrissage à 183 secondes. On voit qu'on atteint une poussée d'environ 15 g (si on enlève le 1 g dû à l'attraction terrestre). La combustion du moteur dure environ 2,5 secondes.

Le graphique suivant fait un zoom sur la poussée du moteur :



La courbe de poussée qui en découle se rapproche beaucoup de la théorie et semble confirmer l'hypothèse des caractéristiques imprévues du propulseur. La courbe obtenue illustre une poussée plus énergétique cette année, en comparaison à l'an passé. On observe aussi les accélérations relatives à l'ouverture de chacun des parachutes ainsi qu'à l'atterrissage.

On peut aussi faire un zoom sur l'ouverture des parachutes :

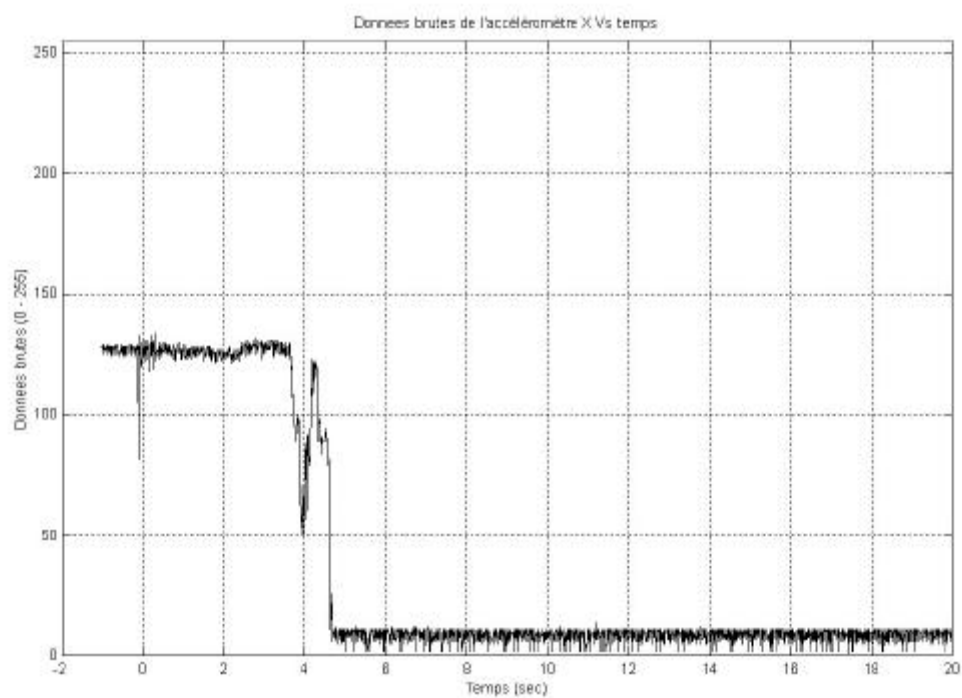
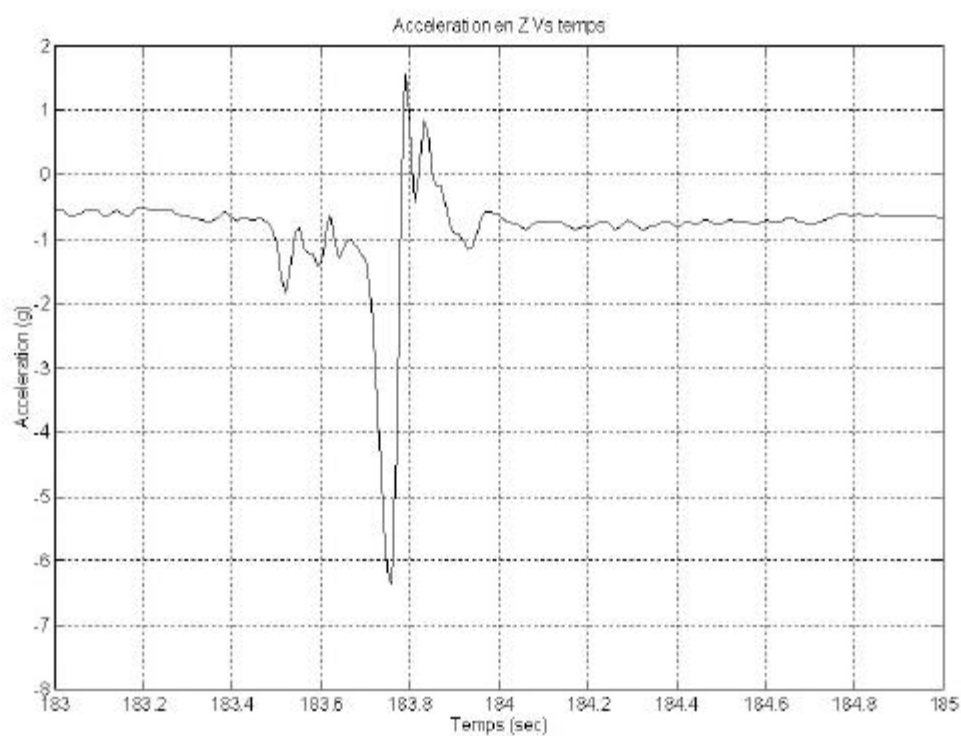


On peut voir que le choc dû au 1^{er} parachute est faible, environ -1,8 g. Celui du parachute principal est plus violent, il fait saturer le capteur à 7,5 g. On peut supposer que le choc réel est supérieur à -8 g. L'ouverture des 2 parachutes était prévue pour une accélération de -4 g. La divergence entre les 2 résultats peut être imputée au vent intense sévissant alors en altitude. On peut apprécier la solidité du second parachute qui a résisté à 2 fois la force à laquelle il était conçu.

Notons que l'accélération avant l'ouverture du 1^{er} parachute est très près de zéro. On peut supposer une vitesse constante et une trajectoire horizontale, précédant l'ouverture, précisant qu'elle a eu lieu très près de l'apogée de la trajectoire.

Le graphique suivant montre en détail le choc à l'atterrissage, on voit qu'il est un peu supérieur à 5 g. Il est intéressant de noter l'accélération juste avant le choc, on voit qu'elle est proche de -0,7 g. C'est en fait la composante de l'attraction de la terre en Z mais inversée, puisque la fusée pointe vers le sol !

Finalement, il ne faut pas oublier que *Babylone* mesurait l'accélération sur 3 axes, et non un seul. Il aurait été intéressant de mettre en commun ces 3 axes pour avoir le module de l'accélération sur la fusée. Malheureusement, l'accéléromètre qui était monté perpendiculaire au PCB a flanché 4 secondes après le décollage. Il a été établi plus tard que c'est une pin du SurfBoard qui s'est cassé à cause de la forte accélération et de la vibration. Le dernier graphique illustre ce problème.



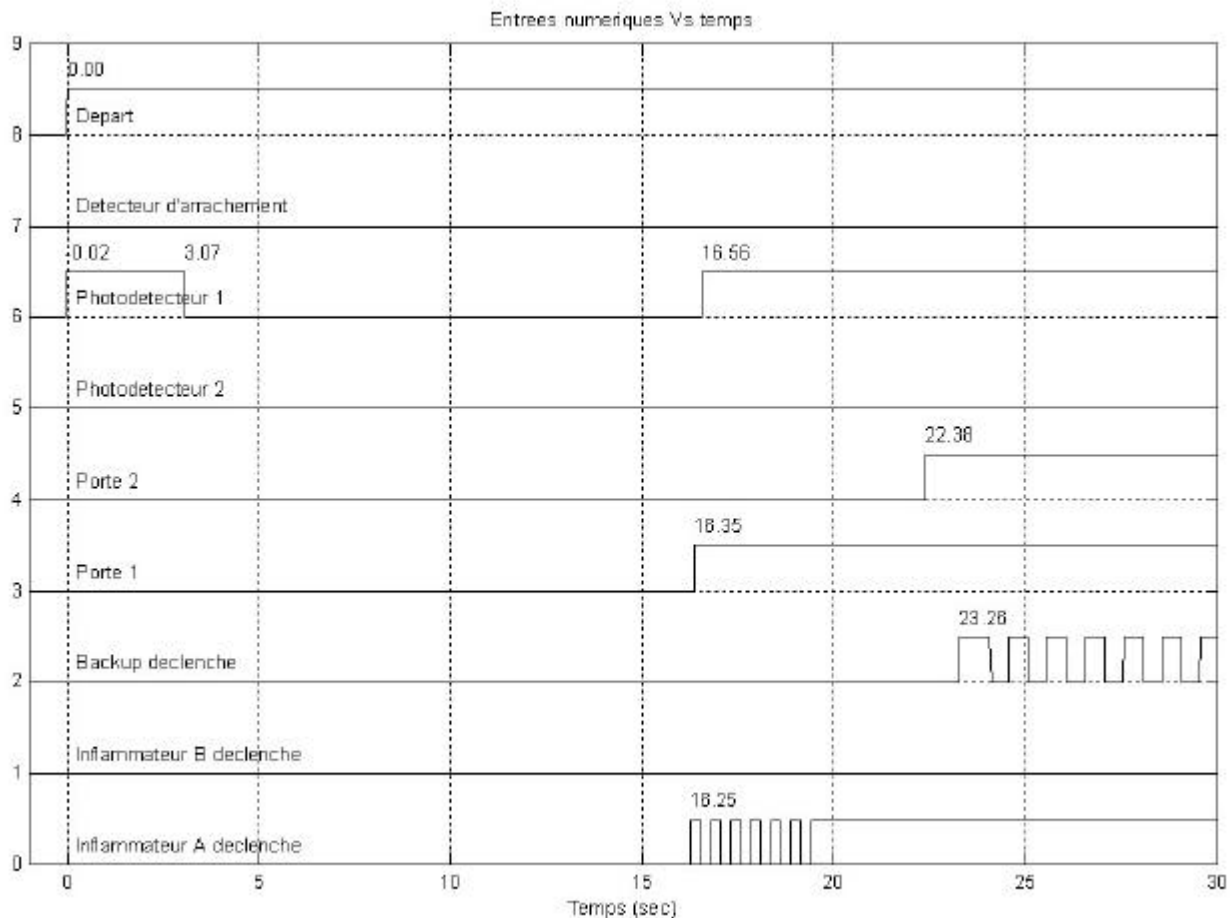
5.3 Données numériques

Le graphique suivant illustre les données et états numériques de *Babylone* durant le vol. Le zéro a été placé pour concorder avec le signal "Départ". Ce signal est déclenché lorsque le fil attaché sur la rampe de lancement arrache. Il faut noter ce signal de départ n'est pas exactement le bon, en effet la fusée a un peu bougé avant que le fil arrache. À l'aide des autres graphiques, on peut estimer ce délai à environ 100 millisecondes, ce qui est à peu près négligeable.

Il est très intéressant d'observer la séquence de déploiement du 1^{er} parachute. Premièrement, le signal d'ouverture est envoyé à 16,25 secondes et le solénoïde se déclenche. On détecte l'ouverture de la porte à 16,35 secondes, soit 100 millisecondes plus tard. Finalement, le photodétecteur au fond de la case parachute détecte que ce dernier est sorti de sa case à 16,56 secondes, soit environ 0,3 secondes après le signal de déclenchement. Le moins qu'on puisse dire est que le parachute est sorti sans hésitation!

L'ouverture du parachute principal quant à elle a été détectée à 22,38 secondes par le détecteur de porte. Le photodétecteur n'a pas marché à cause d'une mauvaise calibration (manque de temps) mais on sait très bien que le parachute est bel et bien sorti! Malheureusement le temps exact du déclenchement de l'ouverture est inconnu. Un problème avec l'opto-coupleur peut-être? Quoi qu'il en soit, le déclenchement a probablement eu lieu à 22 secondes, soit à la fin de la fenêtre de temps du séquenceur. Un peu plus tard et on atteignait le Tbw du backup (23 secondes). En effet, on peut voir que peu après l'ouverture du parachute principal, le backup s'est déclenché. Il a donc fonctionné comme prévu, il n'a pas déclenché l'ouverture mais l'aurait fait en cas de problèmes!

Finalement, on ne note aucune activité sur le détecteur d'arrachement. Normal, le 1^{er} parachute ne s'est pas arraché...



6 Conclusion

Annexe 1

Photographies



Annexe 2

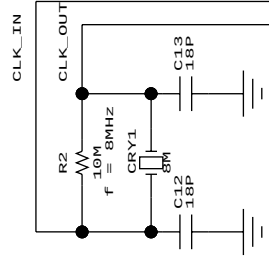
Plans mécaniques

Annexe 3

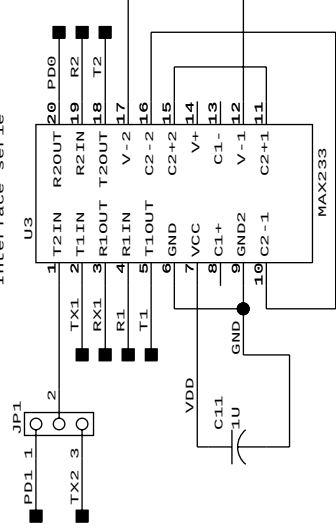
Schémas électroniques

P3

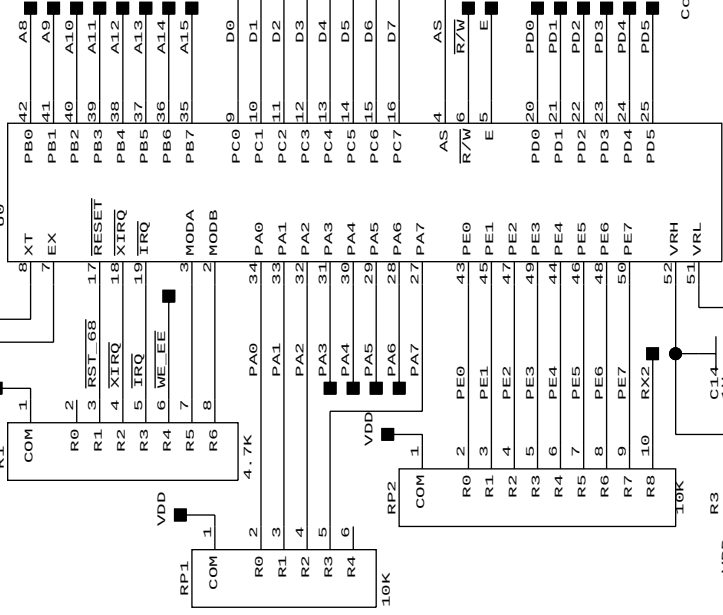
Cond. de découplage pour le 68HC11



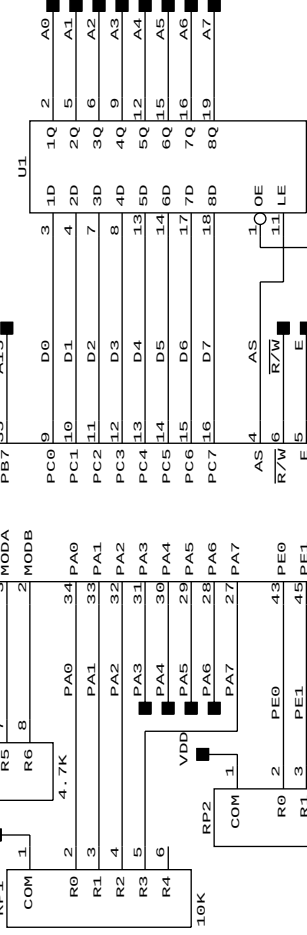
Interface serie



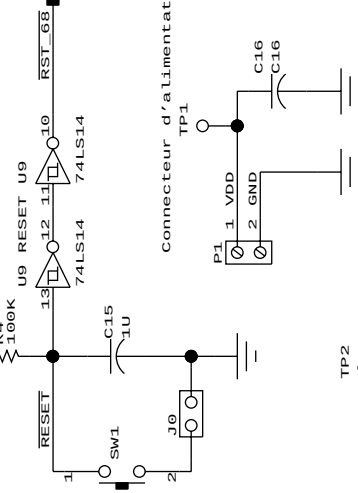
Cerveau de la chose!



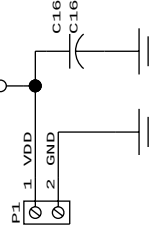
Demultiplexeur d'adresses



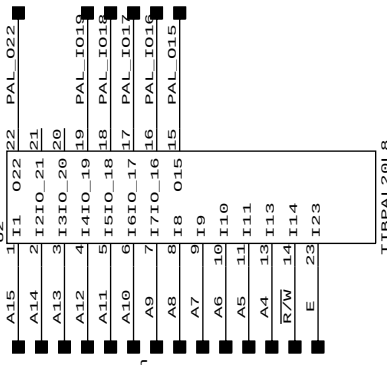
Circuit de reset



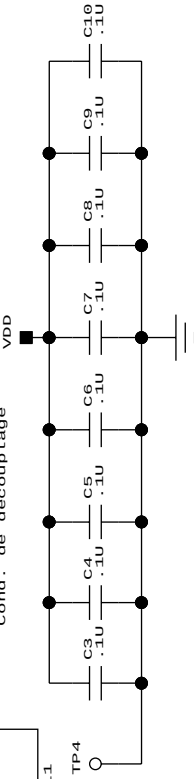
Connecteur d'alimentation



PAL decodage d'adresses

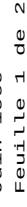
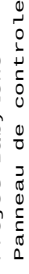
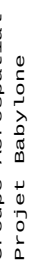
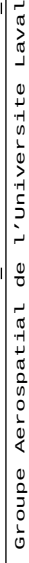
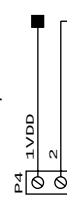
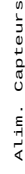
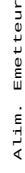
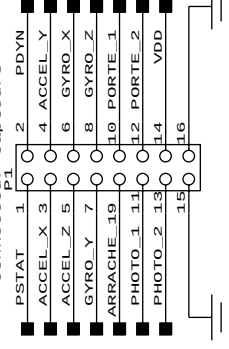
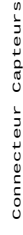
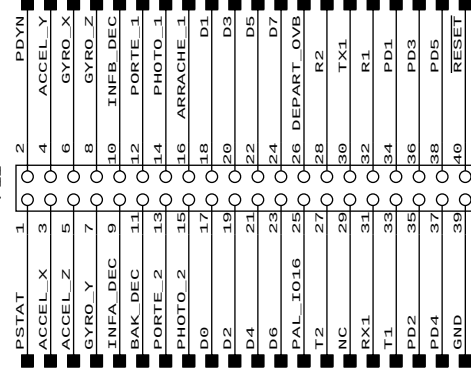
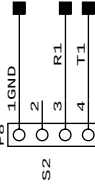
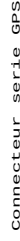
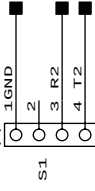
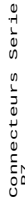


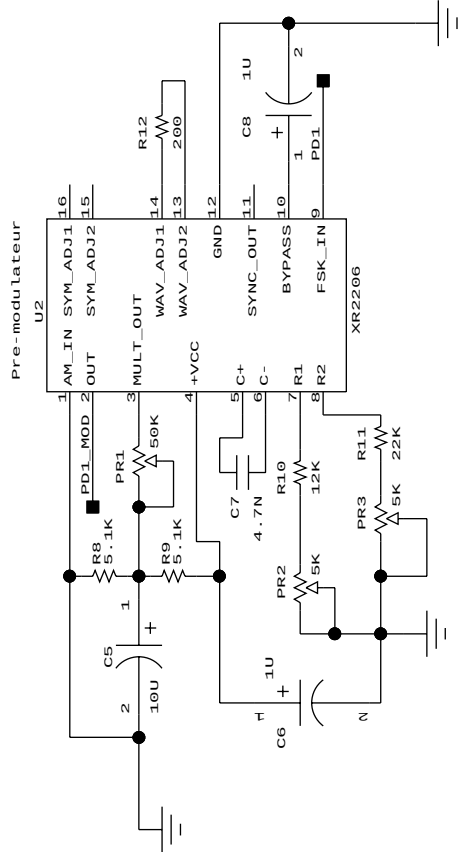
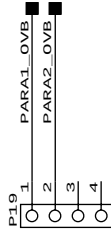
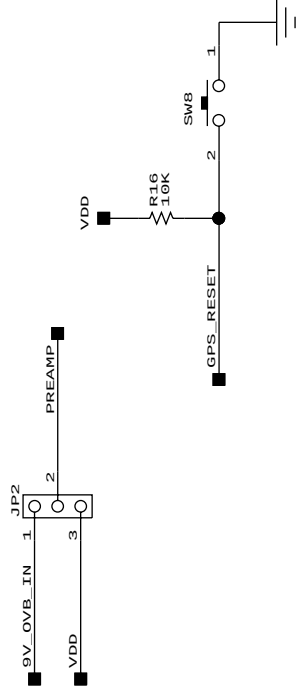
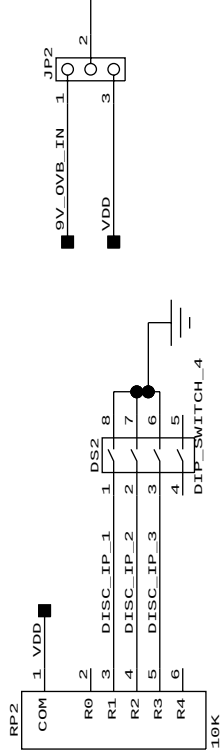
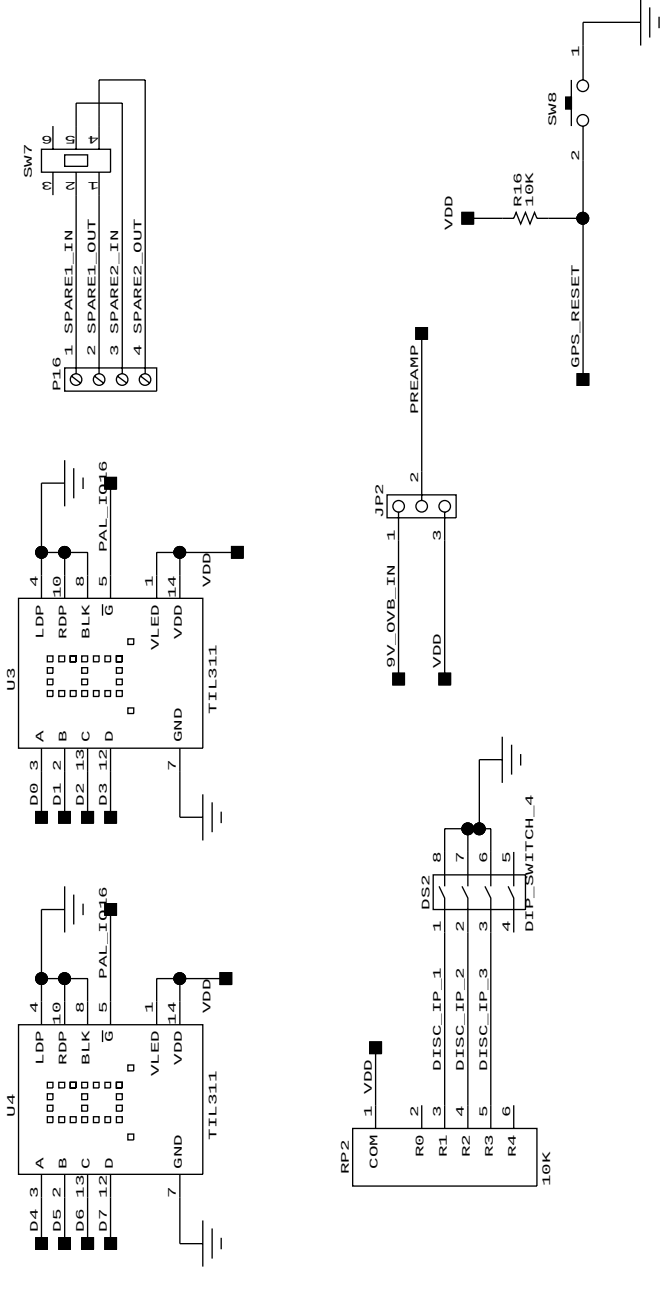
Cond. de découplage



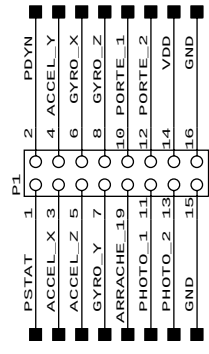
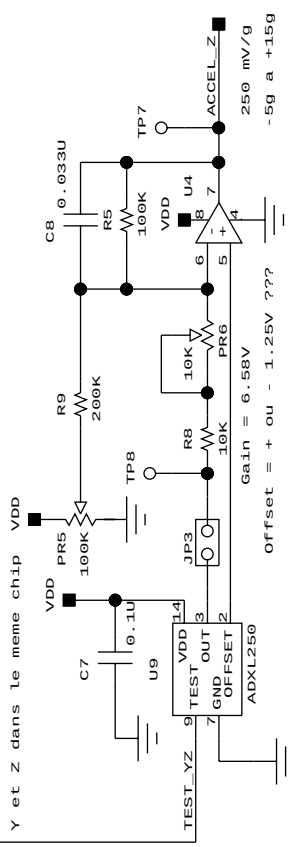
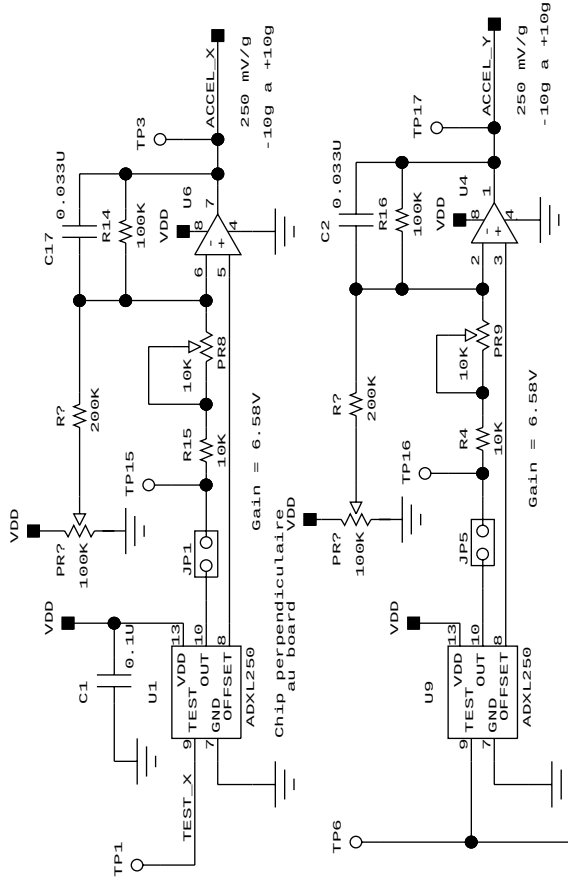
TIBPAL20L8





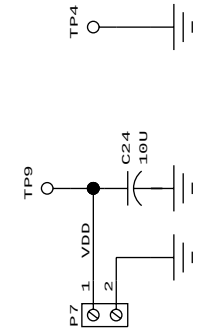


Accelerometres

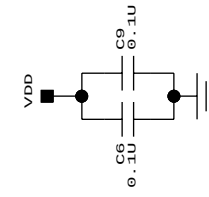


Connecteur du panneau de controle

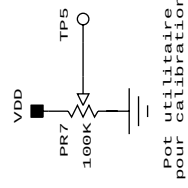
Alimentation



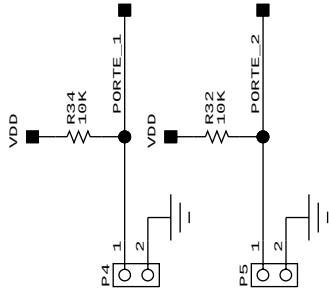
Decouplage pour les OP296



Pot utilitaire pour calibration



Detecteur des portes



Detecteur d'arrachement du 1er parachute

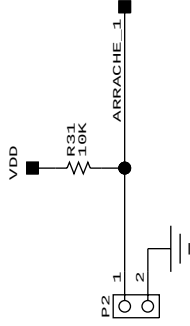
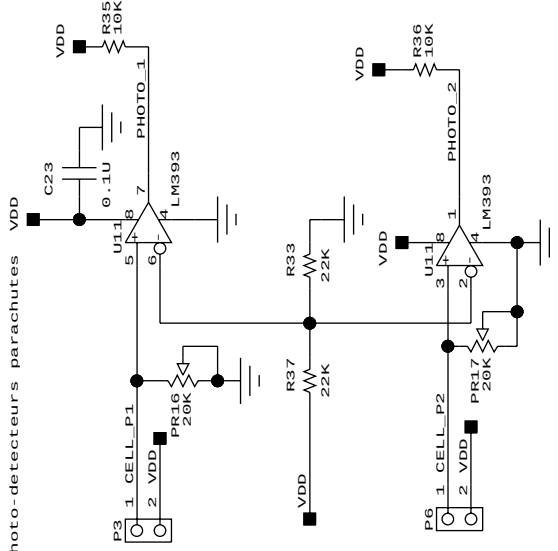
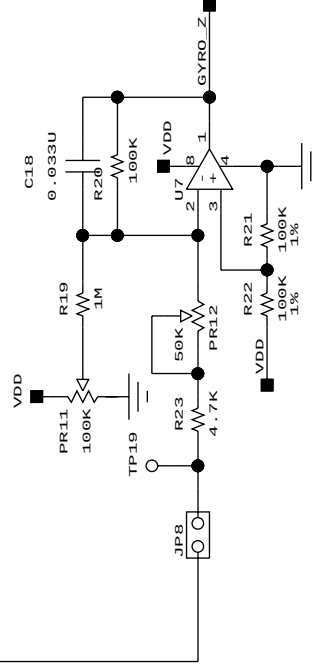
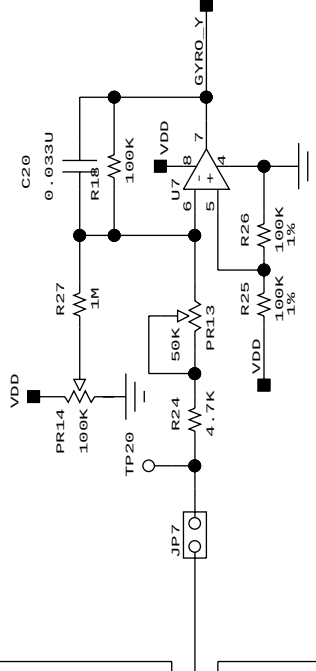
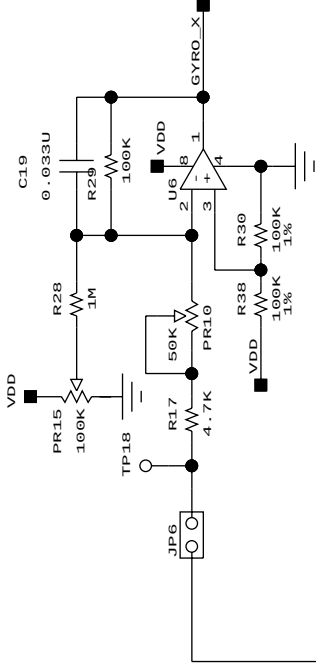
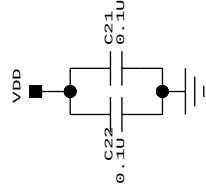
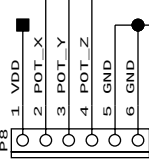
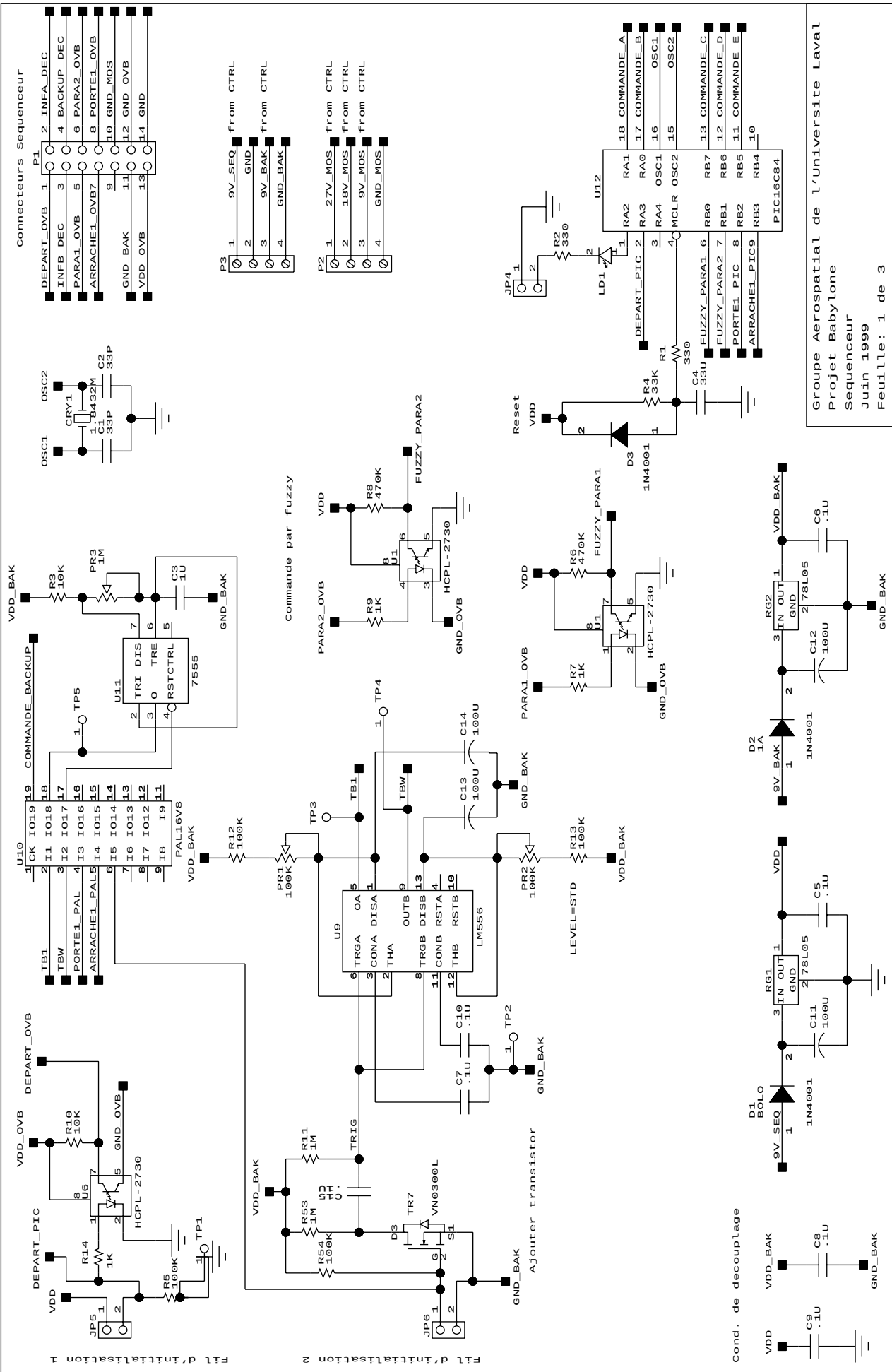


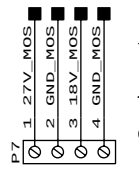
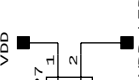
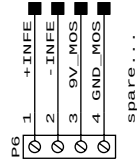
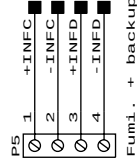
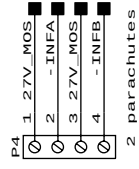
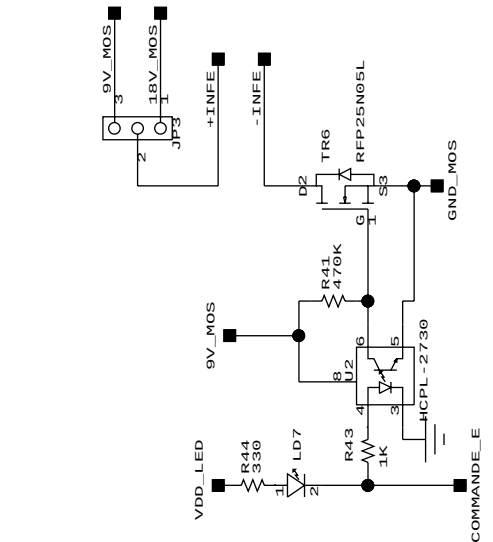
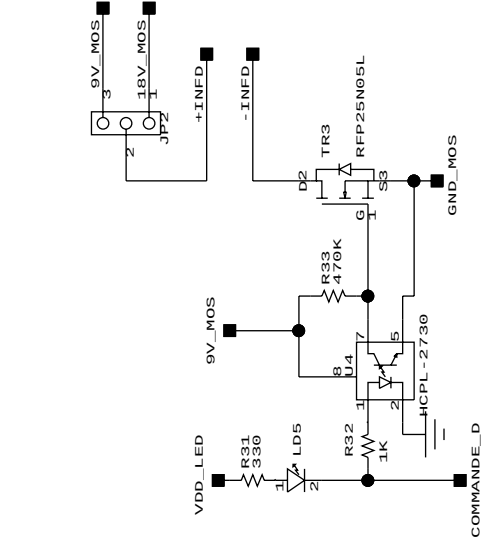
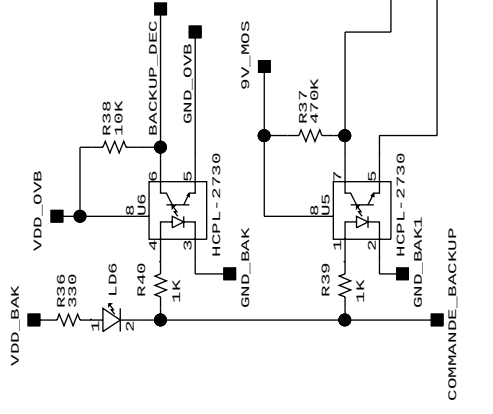
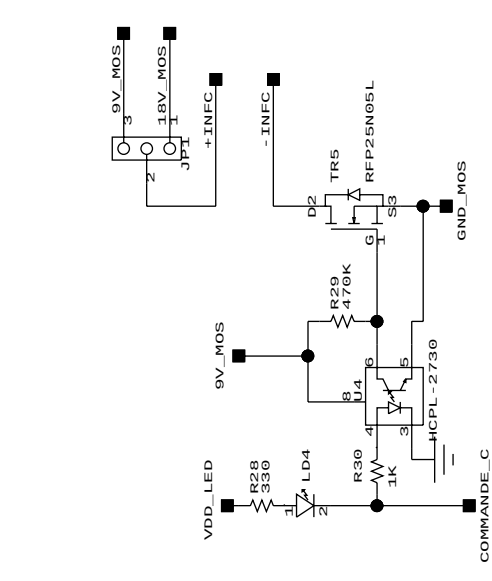
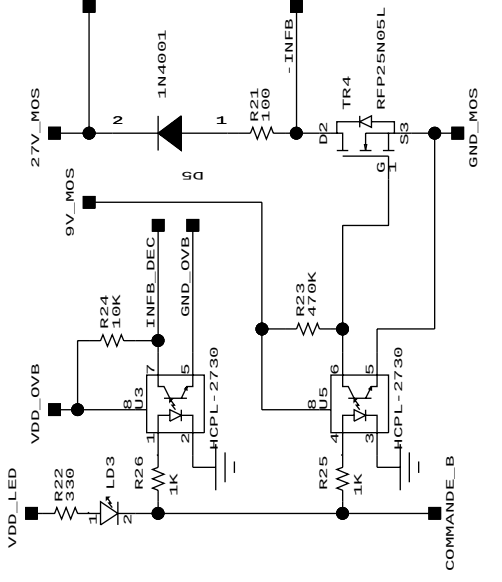
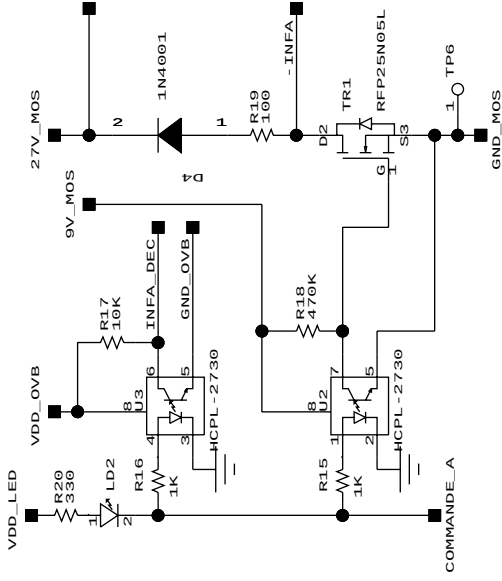
Photo-detecteurs parachutes



Connecteur Gyro





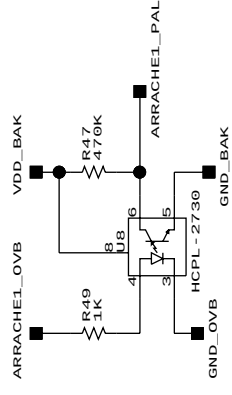
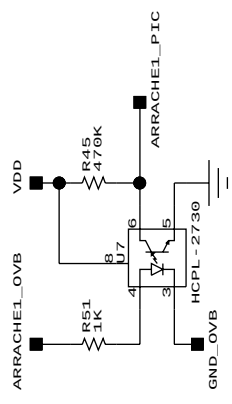
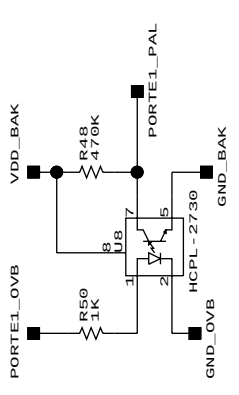
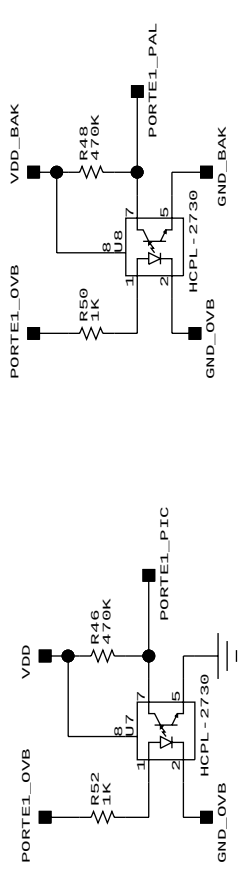


2 parachutes

Fum1.. + backup

spare...

Condensateurs pour puissance



Annexe 4

Code

```
module Adr_dec
```

```
title 'Decodeur d adresse pour la fusée Babylone, GAUL Septembre 1999'
```

```
U3 device 'P20R8AS';
```

```
" Definitions
```

```
" General  
X = .X.;  
H = 1;  
L = 0;
```

```
" Entrees  
A15,A14,A13,A12,A11,A10,A9,A8,A7,A6,A5,A4      pin 1,2,3,4,5,6,7,8,9,10,11,13;  
RW      pin 14;  
E      pin 23;
```

```
" Sorties  
EEPROM      pin 15; " Memoire code  
AFFICHEURS  pin 16; " Afficheurs hexadecimaux  
HADR        pin 17; " buffer adresses hautes  
NUM         pin 18; " Buffer entrees numerique, semblant de port //  
SERIAL      pin 19; " SCC2692, controleur serie DUART  
SRAM        pin 22; " Memoire non-volatile pour les donnees 512k X 8
```

```
" Interne  
adr = [A15,A14,A13,A12, A11,A10,A9,A8, A7,A6,A5,A4, X,X,X,X];
```

```
equations
```

```
!SERIAL = ( adr >= ^h2000 ) & ( adr <= ^h20FF ) & E;  
" 5 lignes d'adresses -> 32 registres internes, 256 de reserves
```

```
HADR = ( adr >= ^h3FE0 ) & ( adr <= ^h3FFF ) & E & !RW;  
" 1 octets situe juste avant la SRAM pour la paginee, lecture seulement
```

```
!SRAM = ( adr >= ^h4000 ) & ( adr <= ^h7FFF ) & E;  
"14 lignes d'adresse standard + 5 de HADR -> 32 pages de ^h4000 octets
```

```
!AFFICHEURS = ( adr >= ^hA100 ) & ( adr <= ^hA10F ) & E & !RW;  
" TIL311 -> 1 octet, 16 de reserves ( le minimum possible ), lecture seulement
```

```
!NUM = ( adr >= ^hA110 ) & ( adr <= ^hA11F ) & E & RW;  
" 74HC373 -> 1 octet, 16 de reserve ( le minimum possible ), lecture seulement
```

```
!EEPROM = ( adr >= ^hC000 ) & ( adr <= ^hFFFF ) & E;  
" 13 lignes d'adresse standard -> plage de ^h2000 octets
```

```
test_vectors ( [ adr, E, RW ] -> [ SERIAL, HADR, SRAM, AFFICHEURS, NUM, EEPROM ] )  
[ X, L, X ] -> [ H, L, H, H, H, H ];  
[ ^h2010, H, X ] -> [ L, L, H, H, H, H ];  
[ ^h3FEF, H, L ] -> [ H, H, H, H, H, H ];  
[ ^h3FEF, H, H ] -> [ H, L, H, H, H, H ];  
[ ^h7000, H, X ] -> [ H, L, L, H, H, H ];  
[ ^hA10C, H, L ] -> [ H, L, H, L, H, H ];  
[ ^hA10C, H, H ] -> [ H, L, H, H, H, H ];  
[ ^hA11C, H, H ] -> [ H, L, H, H, L, H ];  
[ ^hA11C, H, L ] -> [ H, L, H, H, H, H ];  
[ ^hD000, H, X ] -> [ H, L, H, H, H, L ];
```

```
end Adr_dec
```

```
" *****
" ** Fichier: backup.abl
" ** Description: Code du PAL pour le backup du sequenceur sur la fusée Babylone.
" ** Dernière modification: janvier 2000 (ajout de commentaires)
" ** Auteur: Patrick Dubois
" **
" ** Le PAL ouvre seulement le parachute principal, pas le drag. En resume,
" ** il ouvre le main aussitot que le drag arrache OU si on atteint Tb1 et
" ** que le drag n'est pas encore ouvert OU si on se rend a Tbw.
" *****

module BACKUP

pal_16V8 device 'P16V8AS';    " Le AS est pour le mode Simple

" Tb1 est le T2 du sequenceur + X secondes
" Tbw est le timer 'whatever', i.e. on ouvre le parachute 2.
" P est le detecteur d'ouverture de porte du 1er parachute (active low)
" A est le detecteur d'arrachement du 1er parachute (active low)
" D est le signal de decollage (active high)

" ***** ENTREES *****
From556_1    pin 2;           " Passe a high lorsqu'on arrive a Tb1
From556_2    pin 3;           " Passe a high lorsqu'on arrive a Tbw
!P           pin 4;           " Low lorsque la porte est ouverte
!A           pin 5;           " Low lorsque le parachute est arrache
D            pin 6;
From555      pin 18;

" ***** SORTIES *****
CommandBAK   pin 19;          " Doit etre LOW pour declencher l'ouverture
To555        pin 17;

" ***** SORTIES NON BRANCHEES *****
TDV          pin 14;
Tb1          pin 13;
Tbw          pin 12;

" ***** DECLARATIONS *****
Declarations

@dcset

" ***** EQUATIONS *****
Equations

To555 = TDV;                  " Le 555 est en reset lorsque To555 = 0
CommandBAK = !From555;        " Lorsqu'en reset, la sortie du 555 est LOW donc CommandBAK est HIGH
Tb1 = (!From556_1 & D);
Tbw = (!From556_2 & D);

" ***** TABLE DE VERITE *****
truth_table ([Tb1, Tbw, D , P , A ] -> TDV)

    [ .X. ,   .X. , 0 , .X. , .X. ] -> 0 ;    " Pas decolle, on ne fait rien.

    [ 0 ,   0 , 1 , .X. , .X. ] -> 0 ;    " On n'a pas encore atteint Tb1, on ne
                                           " fait rien et on laisse faire le PIC."

    [ 1 ,   0 , 1 , 1 , 0 ] -> 0 ;    " Tout va bien, drag ouvert.

    [ 1 ,   0 , 1 , 1 , 1 ] -> 1 ;    " Le drag est arrache, on ouvre le main!
    [ 1 ,   0 , 1 , 0 , 0 ] -> 1 ;    " Oups, Tb1 atteint mais le drag n'est pas
                                           " encore ouvert, on ouvre le main!
    [ 1 ,   0 , 1 , 0 , 1 ] -> 0 ;    " Le parachute est arrache mais la porte
                                           " n'est pas ouverte??? On ne fait rien, il
                                           " vaut mieux attendre Tbw.
```

```
[ 0 , 1 , 1 , 0 , 0 ] -> 0 ;      " Tbw avant Tb1??? On ne fait rien.
[ 0 , 1 , 1 , 1 , 1 ] -> 1 ;      " Tbw avant Tb1??? Drag arrache? On ouvre
                                   " le main!
[ 0 , 1 , 1 , 1 , 0 ] -> 0 ;      " Tbw avant Tb1??? Le drag est ouvert,
parfait.
[ 0 , 1 , 1 , 0 , 1 ] -> 0 ;      " Tbw avant Tb1??? Drag arrache mais porte
fermee???
                                   " Situation la plus
bizarre... on ne fait rien.

[ 1 , 1 , 1 , .X. , .X. ] -> 1 ; " Advienne que pourra, on ouvre le main!

" ***** TEST VECTORS *****
" Les test vectors doivent etre fait avec les vraies entrees et pas de .X.

test_vectors ([From556_1, From556_2, D , P , A ] -> [TDV, To555])
[ 0 , 0 , 0 , 0 , 0 ] -> [0, 0];
[ 0 , 0 , 0 , 0 , 1 ] -> [0, 0];
[ 0 , 0 , 0 , 1 , 0 ] -> [0, 0];
[ 0 , 0 , 0 , 1 , 1 ] -> [0, 0];
[ 0 , 1 , 0 , 0 , 0 ] -> [0, 0];
[ 0 , 1 , 0 , 0 , 1 ] -> [0, 0];
[ 0 , 1 , 0 , 1 , 0 ] -> [0, 0];
[ 0 , 1 , 0 , 1 , 1 ] -> [0, 0];
[ 1 , 0 , 0 , 0 , 0 ] -> [0, 0];
[ 1 , 0 , 0 , 0 , 1 ] -> [0, 0];
[ 1 , 0 , 0 , 1 , 0 ] -> [0, 0];
[ 1 , 0 , 0 , 1 , 1 ] -> [0, 0];
[ 1 , 1 , 0 , 0 , 0 ] -> [0, 0];
[ 1 , 1 , 0 , 0 , 1 ] -> [0, 0];
[ 1 , 1 , 0 , 1 , 0 ] -> [0, 0];
[ 1 , 1 , 0 , 1 , 1 ] -> [0, 0];

[ 1 , 1 , 1 , 0 , 0 ] -> [0, 0];
[ 1 , 1 , 1 , 0 , 1 ] -> [0, 0];
[ 1 , 1 , 1 , 1 , 0 ] -> [0, 0];
[ 1 , 1 , 1 , 1 , 1 ] -> [0, 0];

[ 0 , 1 , 1 , 1 , 0 ] -> [0, 0];
[ 0 , 1 , 1 , 1 , 1 ] -> [1, 1];
[ 0 , 1 , 1 , 0 , 0 ] -> [1, 1];
[ 0 , 1 , 1 , 0 , 1 ] -> [0, 0];

[ 1 , 0 , 1 , 0 , 0 ] -> [0, 0];
[ 1 , 0 , 1 , 1 , 1 ] -> [1, 1];
[ 1 , 0 , 1 , 1 , 0 ] -> [0, 0];
[ 1 , 0 , 1 , 0 , 1 ] -> [0, 0];

[ 0 , 0 , 1 , 0 , 0 ] -> [1, 1];
[ 0 , 0 , 1 , 0 , 1 ] -> [1, 1];
[ 0 , 0 , 1 , 1 , 0 ] -> [1, 1];
[ 0 , 0 , 1 , 1 , 1 ] -> [1, 1];

test_vectors (From555 -> CommandBAK)
    0 -> 1;
    1 -> 0;
end
```

```
/* **** */
/* Programmation du PIC16C84 (pour le sequenceur de Babylone) */
/* Auteur: Patrick Dubois */
/* Dernière modification: janvier 2000 (ajout de commentaires) */
/* Groupe Aérospatial de l'Université Laval */
/* VERSION FINALE */
/* **** */

#include "SEQ99C.H"
typedef short int bit;

/* Cristal, WatchDog ON, No Code Protect, PowerUp Timer ON */
#fuses XT, WDT, NOPROTECT, PUT

/* inputs */
#define IDECOLLAGE PIN_A3 // Active low
#define iFuzzyP1 PIN_B0 // Active low
#define iFuzzyP2 PIN_B1 // Active low
#define iPORTE1 PIN_B2 // Active low
#define iARRACHE PIN_B3 // Active low
/* outputs */
#define oCOMMANDE_DRAG PIN_A1
#define oCOMMANDE_MAIN PIN_A0
#define oCOMMANDE_C PIN_B7
#define oCOMMANDE_D PIN_B6
#define oCOMMANDE_E PIN_B5
#define oLED_DECOLLAGE PIN_A2

#define RTCC_INTS_PER_SECOND 225 // 1843200/(4*256*8)

#define T1min 16 // Temps minimum avant l'ouverture du drag.
#define T1max 19 // Temps maximum avant l'ouverture du drag.
#define T2min 3 // Temps minimum entre l'ouverture du drag
// du main.
#define T2max 22 // Temps maximum entre l'ouverture du drag.
// et du main.

#define Tfumigenes 3
#define PORTE_DELAY 1 // Temps qu'on laisse a la porte pour s'ouvrir apres
// le signal d'ouverture du drag.

#define GLITCH_DELAY 50 // en ms.

byte t1 = 0; // Compteur de secondes depuis le decollage.
byte t2 = 0; // Compteur de secondes depuis l'ouverture
// du drag.

byte int_t1_count; // Nb d'interrupt restant avant qu'une seconde se
// soit ecoulee pour t1.
byte int_t2_count; // Nb d'interrupt restant avant qu'une seconde se soit ecoulee pour t2

bit DECOLLEE = FALSE; // Flag qui indique si la fusée est decollée
bit DRAG_OPENED = FALSE; // Flag qui indique si le drag chute a été ouvert
bit MUST_OPEN_DRAG = FALSE; // Flag qui indique qu'on doit ouvrir le drag chute
bit MUST_OPEN_MAIN = FALSE; // Flag qui indique qu'on doit ouvrir le main chute

byte STATUS_REGISTER_ADDRESS = 0x03;

/* **** */
** void delai_ms(int _ms)
/* **** */
** DESCRIPTION: Fonction utilitaire qui permet de faire un delai tout en
** restartant le watchdog.
** INPUTS: int _ms : delai en ms
/* **** */
void delai_ms(int _ms) {
    while (_ms > 0)
    {
        restart_wdt();
        delay_ms(1);
        --_ms;
    }
}
```

```
restart_wdt();
}

/*****
** #int_rtcc clock_isr()
*****/
** DESCRIPTION: Cette fonction est appelée chaque fois que le timer0
**              overflow (255->0). Pour une clock de 1,834 MHz et
**              RTCC_DIV_8 c'est environ 225 fois par secondes.
**
** NOTE: Il ne doit pas y avoir trop d'instructions dans cette fonction
**       pcq sinon elle n'aura pas le temps de terminer avant le prochain
**       interrupt.
*****/
#int_rtcc
clock_isr()
{
    if (DECOLLEE == TRUE)
    {
        // On regarde si on doit incrementer t2...
        if (DRAG_OPENED == TRUE)
        {
            if(--int_t2_count==0)
            {
                ++t2;
                int_t2_count = RTCC_INTS_PER_SECOND;
            }
        }
        if(--int_t1_count==0)
        {
            ++t1;
            int_t1_count = RTCC_INTS_PER_SECOND;
        }
    }
}

#int_eeprom // EEPROM write complete
eeprom_isr()
{
}

/*****
** void attend_decollage()
*****/
** DESCRIPTION: Fonction qui boucle sur elle meme en attendant que le
**              signal iDECOLLAGE passe a 0.
**
*****/
void attend_decollage()
{
    while (DECOLLEE == FALSE)
    {
        while (input(iDECOLLAGE) == 1) // On attend le decollage.
        {
            restart_wdt();
        }
        delai_ms(GLITCH_DELAY); // On met un delai pour etre certain
                                // que ce n'est pas un glitch
        if (input(iDECOLLAGE) == 0)
        {
            DECOLLEE = TRUE; // Permet de starter t1
            output_high(oLED_DECOLLAGE);
        }
    }
}
```

```

/*****
** void attend_fumigenes()
*****/
** DESCRIPTION: Fonction qui declenche le fumigene 3 secondes apres le
**               decollage. On laisse la sortie active pendant 2 secondes.
**
*****/
void attend_fumigenes()
{
    while (t1 < Tfumigenes)
    {
        restart_wdt();
    }
    output_low(oCOMMANDE_C);
    delai_ms(250);
    delai_ms(250);
    delai_ms(250);
    delai_ms(250);
    delai_ms(250);
    delai_ms(250);
    delai_ms(250);
    delai_ms(250);
    output_high(oCOMMANDE_C);
}

/*****
** void attend_signal_drag()
*****/
** DESCRIPTION: Fonction qui attend le signal iFuzzyP1 provenant du HC11 et
**               ensuite met le flag MUST_OPEN_DRAG pour permettre de
**               declencher l'ouverture du drag.
**
*****/
void attend_signal_drag()
{
    if (MUST_OPEN_DRAG == FALSE)
    {
        while (t1 < T1max)
        {
            if (input(iFuzzyP1) == 0)
            {
                delai_ms(GLITCH_DELAY);
                if (input(iFuzzyP1) == 0)
                {
                    MUST_OPEN_DRAG = TRUE;          // Permet de démarrer le compteur t2
                    return;
                }
            }
            restart_wdt();
        }
        MUST_OPEN_DRAG = TRUE;
    }
}

/*****
** void ouvre_drag()
*****/
** DESCRIPTION: Fonction qui ouvre le drag avec des pulses et ensuite
**               verifie que le drag n'est pas arrache et que la porte est
**               bien ouverte.
**
*****/
void ouvre_drag()
{
    byte j = 40;
    DRAG_OPENED = TRUE;
    while (t2 < T2min && j > 0) {
        output_low(oCOMMANDE_DRAG);
        delai_ms(250);
        output_high(oCOMMANDE_DRAG);
    }
}
```

```
    delai_ms(250);
    output_low(oCOMMANDE_DRAG);
    --j;
    // On s'assure que le drag n'a pas arrache!
    if (input(iARRACHE) == 0)
    {
        delai_ms(GLITCH_DELAY);
        if (input(iARRACHE) == 0)
        {
            MUST_OPEN_MAIN = TRUE;
            return;
        }
    }
    // On s'assure que la porte est bien ouverte.
    if (t2 > PORTE_DELAY && input(iPORTE1) == 1)
    {
        delai_ms(GLITCH_DELAY);
        if (input(iPORTE1) == 1)
        {
            MUST_OPEN_MAIN = TRUE;
            return;
        }
    }
}

/*****
** void attend_signal_main()
*****/
** DESCRIPTION: Fonction qui attend le signal iFuzzyP2 provenant du HC11 et
**               ensuite met le flag MUST_OPEN_MAIN pour permettre de
**               declencher l'ouverture du parachute principal.
**
*****/
void attend_signal_main()
{
    if (MUST_OPEN_MAIN == FALSE)
    {
        while (t1 < T2max)
        {
            if (input(iARRACHE) == 0)
            {
                delai_ms(GLITCH_DELAY);
                if (input(iARRACHE) == 0)
                {
                    MUST_OPEN_MAIN = TRUE;
                    return;
                }
            }
        }
        if (t2 > T2min)
        {
            if (input(iFuzzyP2) == 0)
            {
                delai_ms(GLITCH_DELAY);
                if (input(iFuzzyP2) == 0)
                {
                    MUST_OPEN_MAIN = TRUE;
                    return;
                }
            }
            if (input(iPORTE1) == 1) // La porte n'est pas ouverte!
            {
                delai_ms(GLITCH_DELAY);
                if (input(iPORTE1) == 1)
                {
                    MUST_OPEN_MAIN = TRUE;
                    return;
                }
            }
        }
    }
} // if (t2 > T2min)
```

```
        restart_wdt();
    } // while
    // On est rendu a T2max alors on doit ouvrir le main chute.
    MUST_OPEN_MAIN = TRUE;
    } // if (MUST_OPEN_MAIN == FALSE)
}

/*****
** void ouvre_main()
*****/
** DESCRIPTION: Fonction qui declenche l'ouverture du parachute principal
**              avec des pulses eternellement!
**
*****/
void ouvre_main() {
    byte i;
    for (i=40; i>0; --i)
    {
        output_low(oCOMMANDE_MAIN);
        delai_ms(250);
        output_high(oCOMMANDE_MAIN);
        delai_ms(250);
        output_low(oCOMMANDE_MAIN);
    }
}

/*****
** main()
*****/
** DESCRIPTION: Boucle principale.
**
*****/
main()
{
    //Pour reperer un restart du au watchdog
    if (restart_cause() == WDT_TIME_OUT)
    {
        bit_clear(STATUS_REGISTER_ADDRESS, 4);
        /*****
        * ON DOIT FAIRE QQCHOSE ICI! (Pas encore implanté...) */
        /*****
    }

    output_high(oCOMMANDE_DRAG);
    output_high(oCOMMANDE_MAIN);
    output_high(oCOMMANDE_C);
    output_high(oCOMMANDE_D);
    output_high(oCOMMANDE_E);
    output_low(oLED_DECOLLAGE);

    setup_counters(RTCC_INTERNAL, RTCC_DIV_8);
    port_b_pullups(FALSE);
    enable_interrupts(RTCC_ZERO);
    //enable_interrupts(INT_EEPROM);
    enable_interrupts(GLOBAL);

    DECOLLEE = FALSE;

    attend_decollage();

    attend_fumigenes();

    while (t1 < T1min)                // On bloque tout signal d'entree
    {
        restart_wdt();
    }

    attend_signal_drag();              // Attend iFuzzyP1 OU T1max et met
                                      // MUST_OPEN_DRAG = TRUE
```

```
ouvre_drag();           // Ouvre le drag chute en produisant des
                        // pulses de 250 ms jusqu'à ce que t2 >=
                        // T2min et met DRAG_OPENED = TRUE

attend_signal_main();    // Attend le signal iFuzzyP2 OU T2max OU
                        // iARRACHE ou iPORTE1 et met MUST_OPEN_MAIN
                        // TRUE.

ouvre_main();           // Ouvre le main chute !!!

while (TRUE)
{
    restart_wdt();
}
}
```